

1. 개요

본 설계 과제는 xv6 파일 시스템에 스토리지 수준의 스냅샷(snapshot / checkpoint) 기능을 확장 구현하는 것을 목표로 한다. 파일 시스템 스냅샷이란 특정 시점의 파일 시스템 전체 상태를 보존하여, 이후 동일 시점으로 복구하거나 읽기 전용으로 참조할 수 있게 하는 기술이며, 이는 프로세스 메모리 상태를 보존하는 프로세스 체크포인팅(checkpointing)과는 구별되는 개념이다.

xv6의 기본 파일 시스템은 스냅샷이나 버전 기반 백업을 지원하지 않기 때문에, 동일 시점의 상태를 유지하려면 디스크 전체를 통째로 복제해야 하며, 이 경우 실제로 변경되지 않은 블록까지 반복적으로 복사되는 비효율이 발생한다. 본 설계에서는 이러한 문제를 해결하기 위해 Copy-on-Write(COW) 기반의 멀티버전 파일 시스템을 도입하고, 각 데이터 블록의 참조 정보를 커널 내부 메타데이터와 온디스크 메타데이터 파일(/snapshot/blockmeta)로 이중 관리하는 구조로 확장하였다.

구체적으로, 모든 디스크 블록에 대해 “현재 파일 시스템에서의 참조 횟수(refcnt)”와 “스냅샷에 의한 참조 횟수(snapcnt)”를 커널 내부의 전역 배열로 관리하고, 부팅 시에는 전체 inode를 스캔하는 rebuild_block_meta()를 통해 이 정보를 재구성한다. 재구성된 참조 정보는 /snapshot/blockmeta 파일에 블록 단위 레코드로 순차 기록되며, 이후 운용 중에는 balloc, bdecref, 스냅샷 생성/삭제 헬퍼 등 모든 블록 참조 변화 지점에서 해당 블록의 레코드를 in-place로 즉시 갱신한다. 이를 통해 스냅샷 관련 시스템 콜 뿐 아니라 일반 파일 연산(write, truncate, unlink 등)까지 포함한 전체 파일 시스템 동작과 블록 메타데이터가 항상 동기화되도록 설계하였다.

스냅샷은 /snapshot/<id> 경로에 읽기 전용(immutable) T_SNAP inode 트리로 생성된다. 스냅샷 생성 시에는 실제 데이터 블록을 복사하지 않고, 기존 데이터 블록을 그대로 공유하되 해당 블록의 refcnt와 snapcnt만 증가시킨다. 이후 현재 파일 시스템에서 어떤 블록에 쓰기 요청이 발생했을 때, 그 블록의 snapcnt > 0이면 스냅샷과 공유 중인 것으로 판단하여 COW를 수행한다. 즉, 새 데이터 블록을 할당하여 기존 내용을 복사한 뒤 inode가 가리키는 블록을 새 블록으로 교체하고, 원래 블록은 참조 카운트를 감소시켜 더 이상 스냅샷이나 파일이 참조하지 않을 경우에만 실제로 해제한다. 스냅샷 삭제 시에는 스냅샷 트리를 재귀적으로 순회하며 snapcnt를 감소시키고, 그 결과 “비트맵상으로는 할당되어 있지만 refcnt == 0 && snapcnt == 0인 고아 데이터 블록”은 별도의 정리 루틴에서 안전하게 해제한다.

이와 같은 구조를 통해 본 설계는 (1) 스냅샷 생성 시 전체 디스크 복제를 하지 않고도 시점 일관성을 보장하고, (2) COW 기반으로 스냅샷과 현재 파일 시스템 간 공간 공유를 극대화하며, (3) 커널 내부 상태와 /snapshot/blockmeta 파일을 일관되게 유지하여, 재부팅·크래시 이후에도 스냅샷 메타데이터를 100% 재구성 가능한 멀티버전 파일 시스템을 구현하였다.

구현한 핵심 사용자 프로그램

snap_create

snap_rollback

snap_delete

meta_dump: /snapshot/metablock(메타데이터 덤프 파일)에 기록되는 데이터인 refcnt(몇 개의 inode가 참조하고 있는지),

snapcnt(몇 개의 스냅샷 inode가 참조하고 있는지)를 ref, snap이라는 값으로 보여주는 프로그램.

사용법: meta_dump

```
$ meta_dump
b 3C : ref=3 snap=2
b 3D : ref=3 snap=2
b 3E : ref=3 snap=2
b 3F : ref=3 snap=2
b 40 : ref=3 snap=2
b 41 : ref=3 snap=2
b 42 : ref=3 snap=2
```

2. 상세 설계

A. block COW 구현 - block COW 기반의 블록 공유 및 해제 정책

A-1. 목표 및 개념 요약

스냅샷이 생성된 시점의 데이터 블록을 변경 없이 보존하려면, 스냅샷과 현재 파일 시스템이 동일한 디스크 블록을 공유해야 저장 공간을 효율적으로 사용할 수 있다. 하지만 공유된 블록에 대해 현재 파일 시스템이 쓰기를 수행한다면, 스냅샷 내부의 과거 내용이 훼손된다. 이를 방지하기 위해 Copy-on-Write(COW) 정책을 적용한다. 구체적으로, 블록이 어떤 스냅샷에서도 참조되지 않는 경우(snapcnt[b]==0)는 기존 블록에 덮어쓰기가 허용된다. 하지만 블록을 하나 이상의 스냅샷이 참조하는 경우(snapcnt[b]>0), COW를 수행한다: 새 블록 할당 -> 기존 데이터 복사 -> inode 매핑 교체.

A-2. 수정/추가한 데이터/함수 구조 정리

전역 배열 및 락

int block_refcnt[FSSIZE]; : 각 디스크 블록 b에 대해, 현재 파일 시스템에서 해당 블록을 참조하는 inode 개수
int block_snapcnt[FSSIZE]; : 블록 b를 참조하는 T_SNAP 타입 inode 개수(스냅샷에서 보호 중인 정도를 나타냄)
struct spinlock block_meta_lock; :block_refcnt / block_snapcnt 배열을 보호하는 스핀락

// 온디스크 메타데이터 파일 구조(각 디스크 블록 b에 대해 아래 구조체가 저장됨

```
struct block_meta {  
    ushort ref;    // block_refcnt[b]  
    ushort snap;  // block_snapcnt[b]  
};
```

관련 함수 및 프로토타입(fs.c)

// refcnt/snapcnt 테이블 초기화 + rebuild_block_meta()

void iinit(int dev);

// 전 inode 스캔하여 ref/snap 카운트 재구성

void rebuild_block_meta(int dev);

// blockmeta 파일의 b번째 레코드를 즉시 갱신

void blockmeta_update_record(uint b);

// refcnt++, (is_snap==1)시 snapcnt++

void blockmeta_add_ref(uint b, int is_snap);

void blockmeta_add_ref_nosnap(uint b);

// snapcnt--(refcnt는 건드리지 않음)

void blockmeta_dec_snap(uint b);

// 새 블록 할당+refcnt=1,snapcnt=0 + blockmeta_update_record()

static uint balloc(uint dev);

// 실제 비트맵 free만 수행하는 함수

static void bfree_raw(uint dev, uint b);

// refcnt-- 후 조건(ref==0&&snap==0)시 bfree_raw() 호출+메타 갱신

static void bdecref(uint dev, uint b);

// snapcnt[b]>0 여부로 COW 필요성 판단

static int need_cow(uint b);

// direct/indirect block 매핑 교체 + old block을 bdecref() 호출

static void set_block(struct inode *ip, uint bn, uint newb);

// 파일 제거 시 모든 블록에 대해 bdecref 호출

void itrunc(struct inode *ip);

// COW write 반영: write 직전 need_cow 판단 -> 새 블록 alloc -> set_block() 호출

int writei(struct inode *ip, char *src, uint off, uint n);

```
// ref==0&&snap==0&&비트맵상 allocated 블록이면 free  
void cleanup_orphan_blocks(int dev);
```

A-3. 구현 함수와 기존 함수의 역할/호출 관계 설명

1. 전역 메타데이터 초기화/재구축 계층

void iinit(int dev)

역할: xv6 파일시스템 초기화. 기존 xv6 역할(슈퍼블록 읽기, inode 캐시 초기화)에 더해:

block_meta_lock 초기화.

rebuild_block_meta(dev)로 메모리 상의 block_refcnt[], block_snapcnt[]를 전부 재계산.

blockmeta_init(dev)로 /snapshot/blockmeta 파일의 inode를 찾아 전역 포인터 blockmeta_ip에 캐시.

호출: 커널 부팅 시 파일시스템 초기화 단계에서 한 번 호출.

void rebuild_block_meta(int dev)

역할: 디스크 상의 모든 inode(dinode) 를 한 번씩 스캔하면서:

block_refcnt[b] = “해당 블록을 참조하는 inode 개수”

block_snapcnt[b] = “해당 블록을 참조하는 T_SNAP inode 개수”

직접/간접 블록 모두 반영.

이 함수는 메모리 상의 refcnt/snapcnt를 “진실”로 재구축하는 역할만 하고, /snapshot/blockmeta 파일은 별도의 헬퍼(blockmeta_write_record 계열) 에 의해 점진적으로(in-place) 갱신된다.

호출: iinit(dev) 끝부분에서 한 번 호출.

필요하다면 snapshot_create(), snapshot_rollback(), snapshot_delete() 같은 스냅샷 관련 코드에서 “전체 재계산”이 필요할 때 다시 호출 가능

static void read_dinode_nolock(int dev, uint inum, struct dinode *out)

역할: 락 없이 디스크에서 inum 번째 dinode를 읽어오는 헬퍼. rebuild_block_meta()에서 inode 전체를 순회할 때, 기존 inode 락/캐시 구조를 건드리지 않고 raw dinode만 읽기 위해 사용.

호출: rebuild_block_meta() 내부.

2. /snapshot/blockmeta 파일 관련 계층

static void blockmeta_init(int dev)

역할: /snapshot 디렉토리에서 blockmeta 엔트리를 찾아 blockmeta_ip 전역 포인터에 저장하는 초기화 함수.

즉, 이후부터는 namei("/snapshot/blockmeta")를 매번 하지 않고 캐시된 inode를 바로 사용할 수 있게 함.

현재 구현에서는 부팅 직후 full sync는 하지 않고,

in-memory block_refcnt[], block_snapcnt[]는 rebuild_block_meta()로 재구성,

디스크의 /snapshot/blockmeta는 이후 balloc(), bdecref(), blockmeta_add_ref() 등에서 필요한 레코드만 부분 갱신하는 구조.

호출: iinit(dev) 마지막에서 한 번 호출.

static void blockmeta_write_record(uint b, int ref, int snap)

역할: 블록 번호 b에 대한 메타데이터 레코드 한 개(ref, snap)를 /snapshot/blockmeta 파일 안의 b * sizeof(struct block_meta) 위치에 in-place로 덮어쓰는 헬퍼.

전제: 이미 바깥에서 begin_op()/end_op() 트랜잭션이 열려 있고,

그 안에서 balloc(), bdecref(), blockmeta_add_ref() 등이 호출된 상황을 가정한다.

그래서 이 함수는 따로 begin_op()를 열지 않고 writei()만 수행.

아직 blockmeta_ip가 준비되지 않은 부팅 극초기에는 (blockmeta_ip == 0) 조용히 리턴해서 크래시를 피함.

호출: balloc()에서 새 블록을 할당한 직후(ref=1, snap=0 저장).

bdecref()에서 refcnt/snapcnt가 변경된 뒤.

blockmeta_add_ref(), blockmeta_dec_snap()에서 ref/snap 변경 후마다.

즉, 메모리 상 카운터가 바뀌는 모든 경로에서 이 함수로 해당 블록의 on-disk 메타데이터를 동기화한다.

void blockmeta_add_ref(uint b, int is_snap)

역할: block_refcnt[b]++ 를 수행하고, is_snap != 0 이면 block_snapcnt[b]++도 같이 수행.

변경된 (ref, snap) 값을 다시 /snapshot/blockmeta의 해당 블록 레코드에 in-place로 기록 (blockmeta_write_record).

일반 파일/디렉토리/스냅샷 inode가 해당 블록을 새로 참조하게 될 때 사용하는 공통 헬퍼.

호출: 스냅샷 계열 구현 코드에서, 예를 들면: snapshot_create()에서 기존 데이터 블록들을 T_SNAP inode에 연결할 때, snapshot_rollback()/snapshot_delete()에서 스냅샷 간 블록 공유 관계를 구축/변경할 때, 즉, “이 블록을 이제부터 한 개 더 참조한다”는 상황에서 호출됨.

void blockmeta_add_ref_nosnap(uint b)

역할: blockmeta_add_ref(b, 0)를 그대로 호출하는 thin wrapper. 즉, 일반 파일/디렉토리용 refcnt++ (snapcnt는 건드리지 않음).

호출: 스냅샷이 아닌 inode에서 블록을 새로 참조하는 코드에서 사용. (예: snapshot API 구현부에서 일반 파일을 복사하거나, refcnt를 별도로 조정하는 경우)

void blockmeta_dec_snap(uint b)

역할: block_snapcnt[b]-- 만 수행하는 함수. refcnt 감소는 여전히 itrunc() → bdecref() 경로에서 처리한다.

스냅샷 inode(T_SNAP)가 없어지거나, 더 이상 어떤 스냅샷도 해당 블록을 참조하지 않을 때

“이 블록이 스냅샷에 의해 보호되는 카운트”만 줄여준다.

변경된 (ref, snap)를 다시 /snapshot/blockmeta에 in-place로 기록.

호출: 예: snapshot_delete()에서 T_SNAP inode가 참조하던 블록들에 대해 먼저 blockmeta_dec_snap()로 snapcnt 감소, 이후 itrunc()에 의해 refcnt가 줄어들고, 최종적으로 bdecref()가 실제 free 여부를 결정.

3. 블록 할당/해제 및 참조 카운트 계층

static uint balloc(uint dev)

역할: 비트맵에서 free 블록 하나를 찾아 할당 + 0으로 초기화.

새로 할당된 블록에 대해:

block_refcnt[bno]를 1로 세팅 (현재 이 블록은 “한 inode만 단독 소유”).

block_snapcnt[bno]를 0으로 세팅.

즉, COW/스냅샷 관점에서 “일반 블록 1개 새로 생성”.

이 ref/snap 값을 blockmeta_write_record()를 통해 /snapshot/blockmeta에도 바로 반영.

호출: bmap()에서 새 데이터/간접 블록이 필요할 때. COW 시, writei()/set_block()에서 새 블록을 할당할 때.

static void bfree_raw(int dev, uint b)

역할: 비트맵에서 해당 블록을 free 처리하는 low-level 함수. refcnt/snapcnt는 건드리지 않고, 오직 디스크 비트맵만 조작. “진짜로 이 블록을 더 이상 아무도 참조하지 않는다”라고 판정된 뒤에만 호출되어야 함.

호출: bdecref()에서 ref == 0 && snap == 0인 경우에만 호출. 즉, 참조 카운트 기반 회수의 마지막 단계.

static void bdecref(int dev, uint b)

역할: block_refcnt[b]-- 를 수행하는 함수. 감소 후 (ref, snap)를 blockmeta_write_record()로 /snapshot/blockmeta에 기록. ref == 0 && snap == 0이면 bfree_raw()를 불러 실제 디스크 비트맵에서도 free.

스냅샷이 공유 중인 블록은 snapcnt > 0이므로, ref가 0이어도 실제 free되지 않고 스냅샷에 의해 핀(pin) 되어 있는 상태를 유지.

호출: itrunc()에서 파일의 direct/indirect 블록을 정리할 때.

COW 후, old 블록을 더 이상 이 inode가 쓰지 않게 되었을 때(writei() / set_block() 이후) 참조 감소용으로 호출.

4. COW 판별 및 블록 매핑 변경 계층

static int need_cow(uint b)

역할: 블록 번호 b에 대해 block_snapcnt[b] > 0인지 확인. 스냅샷 inode(T_SNAP)가 하나라도 해당 블록을 참조 중이면 1을 리턴 → COW 필요. 그렇지 않으면 0 → 그냥 덮어써도 됨.

호출: writei()에서 데이터 블록 덮어쓰기 전에. set_block()에서 간접 블록(ib)에 대한 COW가 필요한지 판단할 때.

static void set_block(struct inode *ip, uint bn, uint newb)

역할: inode ip의 bn번째 논리 블록이 실제로 가리키는 디스크 블록을 newb로 교체.

direct/indirect 모두 처리: bn < NDIRECT: ip->addrs[bn] = newb 후 iupdate(ip).

bn >= NDIRECT: 간접 블록 인덱스로 변환 후, 간접 블록 엔트리 수정.

간접 블록에 대해서는 간접 블록 자체도 스냅샷에 의해 보호될 수 있으므로,

need_cow(ib)를 호출하여 COW가 필요하다면 간접 블록도 새로 할당 + 복사 + 매핑 교체 + bdecref(ib) 수행.

호출: writei()에서 COW 발생 시, 기존 데이터 블록 bno를 새 블록 newb로 복사한 뒤, 해당 논리 블록 bn의 매핑을 교체할 때. (간접 블록에 대해서도 동일하게 사용.)

5. 파일 내용 해제/쓰기 계층

void itrunc(struct inode *ip)

역할: inode의 데이터/간접 블록을 모두 해제(truncate)하는 함수. 기존 xv6의 bfree() 호출을 bdecref()로 교체하여: block_refcnt[]를 감소시키고, ref == 0 && snap == 0일 때만 실제로 비트맵 free. 간접 블록 엔트리 (NINDIRECT) 각각에 대해 bdecref() 호출, 간접 블록 자체도 bdecref()로 처리. 마지막에 ip->size = 0, iupdate(ip)로 inode 크기 0으로 반영.

호출: iput()에서 링크 수와 ref가 모두 0이 된 inode를 정리할 때 호출.

int writei(struct inode *ip, char *src, uint off, uint n)

역할: 파일에 데이터를 쓰는 핵심 함수에 COW 로직을 추가한 버전. 각 블록 쓰기 전에:

bn = off / BSIZE로 논리 블록 계산. bno = bmap(ip, bn)으로 실제 디스크 블록 번호 획득(필요 시 새 블록 할당: balloc() → ref=1). need_cow(bno)가 true이면: newb = balloc()으로 새 블록 할당(ref=1).

bno의 내용을 newb로 memmove()로 복사 후 log_write(). set_block(ip, bn, newb)로 inode 매핑을 newb로 교체. bdecref(ip->dev, bno)로 old 블록에 대한 참조 감소. 이후 쓰기 작업은 새 블록 newb를 대상으로 수행.

해당 블록에 실제 데이터 복사 (memmove + log_write). 쓰기 이후, 파일 크기가 늘어났다면 ip->size 갱신 + iupdate(ip).

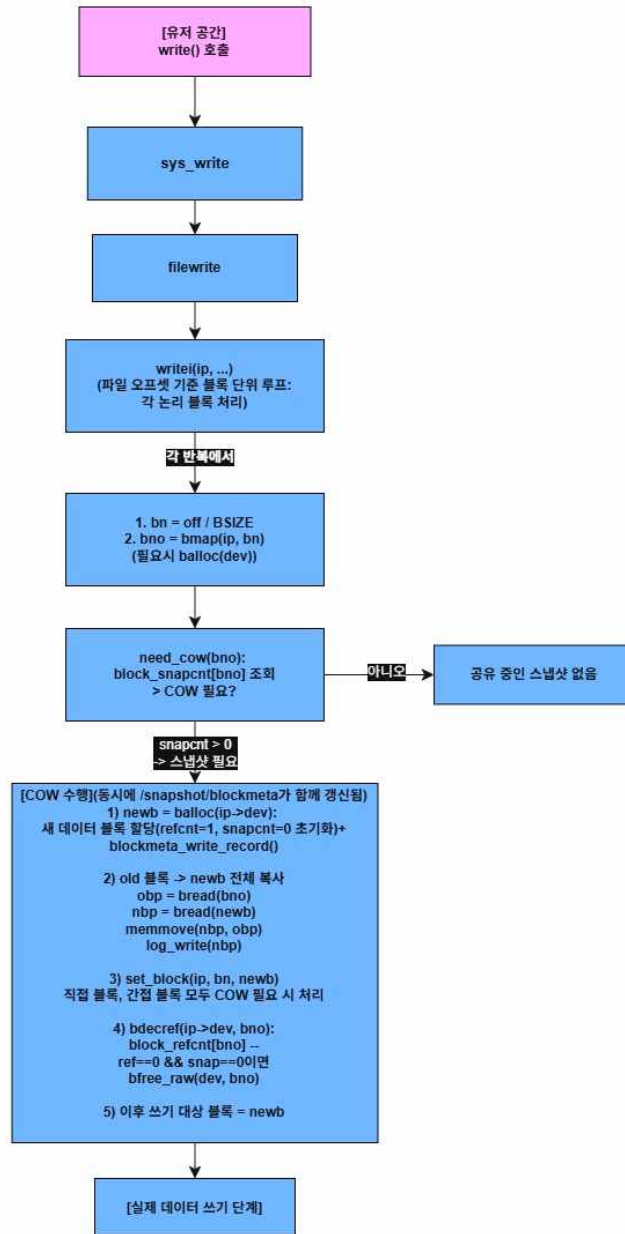
호출: 모든 파일/디렉토리 쓰기 경로에서 공통으로 사용: sys_write(), dirlink(), create() 등.

A-4. Block COW 핵심 동작 흐름도(쓰기 경로)

아래 A. block COW의 흐름도는 어떤 파일에 대한 쓰기 연산이 발생했을 때, xv6 내부에서 block COW 정책과 블록 메타데이터 갱신이 어떤 함수 호출 경로를 통해 수행되는지를 보여준다. 유저 공간에서 write()가 호출되면 커널에서는 sys_write()와 filewrite()를 거쳐 writei()에 도달하며, writei()는 파일 오프셋을 기준으로 블록 단위 루프를 돌면서 각 논리 블록에 대해 bmap()을 호출하여 실제 디스크 블록 번호를 얻는다. 이때 아직 할당되지 않은 논리 블록은 balloc()을 통해 새 블록을 할당하는데, 이 과정에서 비트맵 갱신뿐만 아니라 in-memory block_refcnt[], block_snapcnt[]를 초기화하고, blockmeta_write_record()를 통해 /snapshot/blockmeta 파일의 해당 레코드를 즉시 in-place로 갱신한다.

각 블록에 대해 need_cow()는 block_snapcnt[b]를 조회하여 해당 블록이 하나 이상의 스냅샷(T_SNAP inode)과 공유 중인지 검사한다. snapcnt == 0이면 COW 없이 그대로 덮어쓰고, snapcnt > 0이면 COW 분기로 들어가 balloc()으로 새 블록을 할당한 뒤 기존 내용을 통째로 복사한다. 그 후 set_block()을 호출하여 inode의 블록 매핑을 새 블록으로 교체하는데, 이때 간접 블록이 스냅샷과 공유 중인 경우 need_cow(ib)를 통해 간접 블록 자체에 대해서도 COW를 수행한다. 즉, 간접 블록이 보호 중이면 새로운 간접 블록을 balloc()으로 할당하고, 내용을 복사한 뒤 inode의 간접 블록 포인터를 새 블록으로 바꾸고, 이전 간접 블록에 대해서는 bdecref()를 호출한다.

COW 이후에는 기존 데이터 블록에 대해 bdecref()가 호출되어 block_refcnt[]를 감소시키고, 감소된 (ref, snap) 값은 다시 blockmeta_write_record()를 통해 /snapshot/blockmeta에 반영된다. 이때 ref == 0 && snap == 0이면 더 이상 어떤 파일이나 스냅샷도 해당 블록을 참조하지 않는다는 의미이므로, bfree_raw()를 통해 비트맵에서 블록을 완전히 해제한다. 이렇게 해서 in-memory 카운터와 on-disk 메타데이터 파일이 항상 함께 갱신되는 구조가 된다. 마지막으로, COW 여부와 상관없이 최종적으로 선택된 블록에 대해 데이터를 기록하고 log_write()로 로그 시스템에 반영함으로써, block COW 정책 하에서 스냅샷이 보호하는 블록과 일반 파일 쓰기를 안전하게 분리하면서도 /snapshot/blockmeta 파일 역시 항상 최신 상태에 가깝도록 유지된다.



B. 스냅샷 구현

B-1. 전체 구조 및 핵심 기능

스냅샷 기능은 /snapshot 디렉토리 아래에 다음과 같은 구조로 구현된다.

/snapshot/<id> : 각 스냅샷의 루트 디렉토리(파일 데이터 트리)

/snapshot/blockmeta : 파일 시스템 전체 블록에 대한 refcnt / snapcnt 정보를 저장하는 메타데이터 조회용

파일(/snapshot/blockmeta의 내용은 meta_dump 사용자 프로그램으로 확인 가능하다)

스냅샷 관련 핵심 기능은 다음 세 가지이다.

(요약)

snapshot_create(): 현재 파일 시스템 -> /snapshot/<id>로 캡처

snapshot_rollback(id): /snapshot/<id> -> 현재 파일 시스템으로 복구

snapshot_delete(id): /snapshot/<id> 트리 삭제 + 고아 블록 정리

(더 풀어서 설명)

snapshot_create()

현재 파일 시스템 루트(/)의 내용을 캡처하여 /snapshot/<id> 트리로 복사한다.

원본 파일(T_FILE)은 그대로 두고, 스냅샷 측에는 동일한 블록을 바라보는 스냅샷 파일(T_SNAP)을 만든 뒤 각 블록의 refcnt++, snapcnt++ 를 수행하여 “스냅샷이 이 블록을 보호하고 있음”을 기록한다.

snapshot_rollback(id)

현재 루트(/)에서 /snapshot 디렉토리를 제외한 모든 엔트리를 삭제하고, /snapshot/<id> 트리를 다시 루트 아래로 복원한다.

이때 스냅샷 파일(T_SNAP)은 일반 파일(T_FILE)로 복구되며, 기존 블록을 그대로 공유하도록 refcnt++만 수행한다(snapcnt는 유지). 결과적으로, 복구된 파일은 여전히 해당 스냅샷과 블록을 공유하고 있으므로, 이후 쓰기 시에는 Block COW 로직을 통해 안전하게 분기된다.

snapshot_delete(id)

/snapshot/<id> 트리 전체를 재귀적으로 삭제하면서, 스냅샷 파일(T_SNAP)이 참조하던 블록들에 대해 snapcnt--를 수행한다.

삭제 후에는 cleanup_orphan_blocks()를 통해 refcnt == 0 && snapcnt == 0 이지만 비트맵에는 여전히 할당된 것으로 남아 있는 고아 블록을 실제로 bfree() 한다.

B-2. 수정/추가한 데이터 구조 및 헬퍼

(1) 스냅샷 ID/루트 관리

static struct spinlock snap_lock;

스냅샷 ID 할당용 락. 여러 CPU에서 동시에 snapshot_create()를 호출해도 next_snap_id가 충돌하지 않도록 보호한다.

static int snap_inited;

스냅샷 서브시스템 초기화 여부 플래그.

init_snap_once() 내부에서 한 번만 snap_lock을 초기화하기 위해 사용한다.

static int next_snap_id;

다음에 배정할 스냅샷 ID 값.

snapshot_create()가 호출될 때마다 snap_lock 보호 하에 증가한다.

static void init_snap_once(void);

snap_lock 및 관련 상태를 한 번만 초기화하는 함수.

스냅샷 syscall들이 직접 init_snap_once()를 호출한다.

static int kern_mkdir(char *path);

커널 내부에서 사용하는 mkdir 헬퍼.

begin_op()/end_op()와 create() 호출을 감싸서, /snapshot, /snapshot/<id> 디렉토리를 생성할 때 재사용한다.

static int ensure_snapshot_root(void);

/snapshot 디렉토리가 없으면 kern_mkdir("/snapshot")로 생성하고, 이미 존재하면 그대로 반환한다.

모든 스냅샷 연산의 전제 조건으로 사용된다.

(2) 스냅샷 핵심 API

int snapshot_create(void);

/snapshot 루트를 보장한 뒤, 새로운 ID(/snapshot/<id>)를 생성하고 현재 루트(/) 내용을 재귀적으로 복제한다.

복제 과정에서 원본 T_FILE → 스냅샷 측 T_SNAP으로 대응되며, inc_snap_for_inode_blocks()를 통해 해당 파일이 참조하는 모든 블록에 대해 (refcnt++, snapcnt++) 및 /snapshot/blockmeta 갱신을 수행한다.

int snapshot_rollback(int id);

/snapshot/<id> 스냅샷의 내용을 현재 루트로 복구하는 함수. clear_root_except_snapshot()로 기존 루트에서 snapshot 디렉토리를 제외한 모든 엔트리를 삭제한 후, rollback_clone_dir()를 통해 /snapshot/<id>의 디렉토리/스냅샷 파일을 일반 디렉토리(T_DIR) / 일반 파일(T_FILE)로 새로 생성한다. 복구된 T_FILE들은 기존 스냅샷과 같은 블록을 공유하므로, inc_ref_for_inode_blocks()를 통해 블록마다 refcnt++만 수행하고 snapcnt는 유지한다.

int snapshot_delete(int id);

/snapshot/<id> 트리 전체를 제거하는 함수. snapshot_remove_tree()가 디렉토리 내부를 재귀적으로 비우면서, T_SNAP 파일에 대해서는 dec_snap_for_inode_blocks()를 먼저 호출하여 해당 블록들의 snapcnt--를 수행한다. 이후 /snapshot 디렉토리에서 <id> 엔트리를 dir_unlink_at()으로 제거한 뒤, cleanup_orphan_blocks(ROOTDEV)를 호출하여 (refcnt == 0 && snapcnt == 0) 이면서 비트맵상 할당된 블록들을 실제로 bfree() 한다. 이 과정에서도 ref/snap 변경 시마다 /snapshot/blockmeta의 해당 레코드가 in-place로 갱신된다

(3) 블록 메타데이터 연동(스냅샷 측 헬퍼)

스냅샷 API는 fs 계층에서 제공하는 blockmeta 관련 헬퍼와 연동하여

블록 참조 정보와 /snapshot/blockmeta 파일을 동시에 갱신한다.

(fs.c에서 제공되는 외부 헬퍼)

extern void blockmeta_add_ref(uint b, int is_snap);

: block_refcnt[b]++ 후, is_snap != 0이면 block_snapcnt[b]++까지 수행하고, /snapshot/blockmeta의 해당 레코드를 in-place로 갱신한다. 스냅샷 생성 시(T_SNAP가 새로 블록을 가리킬 때) 사용된다.

extern void blockmeta_add_ref_nosnap(uint b);

일반 파일(T_FILE)이 기존 블록을 함께 쓰게 되는 경우에 사용.

block_refcnt[b]++만 수행하고 block_snapcnt는 건드리지 않으며,

마찬가지로 /snapshot/blockmeta 레코드를 갱신한다.

롤백 시 복구된 파일이 기존 스냅샷과 블록을 공유할 때 쓰인다.

extern void blockmeta_dec_snap(uint b);

스냅샷 파일이 사라질 때, 해당 블록에 대해 block_snapcnt[b]--만 수행하고 /snapshot/blockmeta를 갱신한다.

실제 블록 해제 여부는 refcnt까지 0이 되었을 때 bdecref() 및 cleanup_orphan_blocks()에 의해 결정된다.

(sysfile.c 내부에서 inode 단위로 사용하는 헬퍼)

static void inc_snap_for_inode_blocks(struct inode *ip);

인자로 받은 T_SNAP inode가 참조하는 모든 데이터 블록 및 간접 블록에 대해 blockmeta_add_ref(b, 1)를 호출하여 (refcnt++, snapcnt++)를 수행한다.

snapshot_create()에서 새로 만든 스냅샷 파일에 대해 호출된다.

static void dec_snap_for_inode_blocks(struct inode *ip);

T_SNAP inode가 참조하던 모든 블록에 대해 blockmeta_dec_snap(b)를 호출하여 snapcnt--만 수행한다.

스냅샷 트리를 삭제할 때(snapshot_remove_tree() 경로) 호출되며, refcnt 감소는 itrunc() → bdecref()에서 처리한다.

static void inc_ref_for_inode_blocks(struct inode *ip);

T_FILE inode가 참조하는 모든 블록에 대해 blockmeta_add_ref_nosnap(b)를 호출하여 refcnt++만 수행한다.

snapshot_rollback()에서 스냅샷 파일(T_SNAP)을 일반 파일(T_FILE)로 복구하는 과정에서 사용되며, 스냅샷이 여전히 존재하는 동안에도 COW 조건(refcnt>1, snapcnt>0)을 유지하기 위한 역할을 한다.

(4) 디렉토리/트리 조작 및 unlink 헬퍼

static struct inode *create_subdir(struct inode *parent, char *name);

parent(이미 ilock 상태) 아래에 새 디렉토리(T_DIR)를 만들고 ".", ".." 엔트리를 설정한 뒤 parent에 dirlink() 한다.

static struct inode *create_file(struct inode *parent, char *name);

parent 아래에 새 일반 파일(T_FILE)을 생성한다. 롤백 시 T_SNAP → T_FILE 생성에 사용된다.

static struct inode *create_snapshot_file(struct inode *parent, char *name);

parent 아래에 스냅샷 파일(T_SNAP) inode를 생성하고 dirlink() 한다. 이후 원본 파일의 size와 addrs[]를 복사하고 inc_snap_for_inode_blocks()를 통해 메타데이터를 갱신하는 형태로 사용된다.

static int snapshot_clone_dir(struct inode *src_dir, struct inode *dst_dir, int is_root);

스냅샷 생성 시, / → /snapshot/<id> 복사를 담당. 루트에서만 "snapshot" 디렉토리를 스킵하며, 디렉토리는 create_subdir(), 파일은 create_snapshot_file() + inc_snap_for_inode_blocks()로 처리한다.

static int rollback_clone_dir(struct inode *src_snap_dir, struct inode *dst_dir);

롤백 시 /snapshot/<id> → / 복사를 담당. 스냅샷 디렉토리는 일반 디렉토리로, 스냅샷 파일(T_SNAP)은 일반 파일(T_FILE)로 복구하면서 inc_ref_for_inode_blocks()로 refcnt만 증가시킨다.

static int snapshot_remove_tree(struct inode *dir);

/snapshot/<id> 트리 내부를 재귀적으로 삭제하는 헬퍼. T_SNAP 파일을 만나면 먼저 dec_snap_for_inode_blocks()로 snapcnt를 줄인 뒤 dir_unlink_at()을 통해 디렉토리 엔트리 및 nlink를 정리한다.

static int clear_root_except_snapshot(void);

롤백 시 현재 루트에서 ".", "..", "snapshot"을 제외한 모든 엔트리를 제거한다.

일반 디렉토리는 snapshot_remove_tree()로 내부를 비우고, 이후 dir_unlink_at()으로 엔트리를 제거한다.

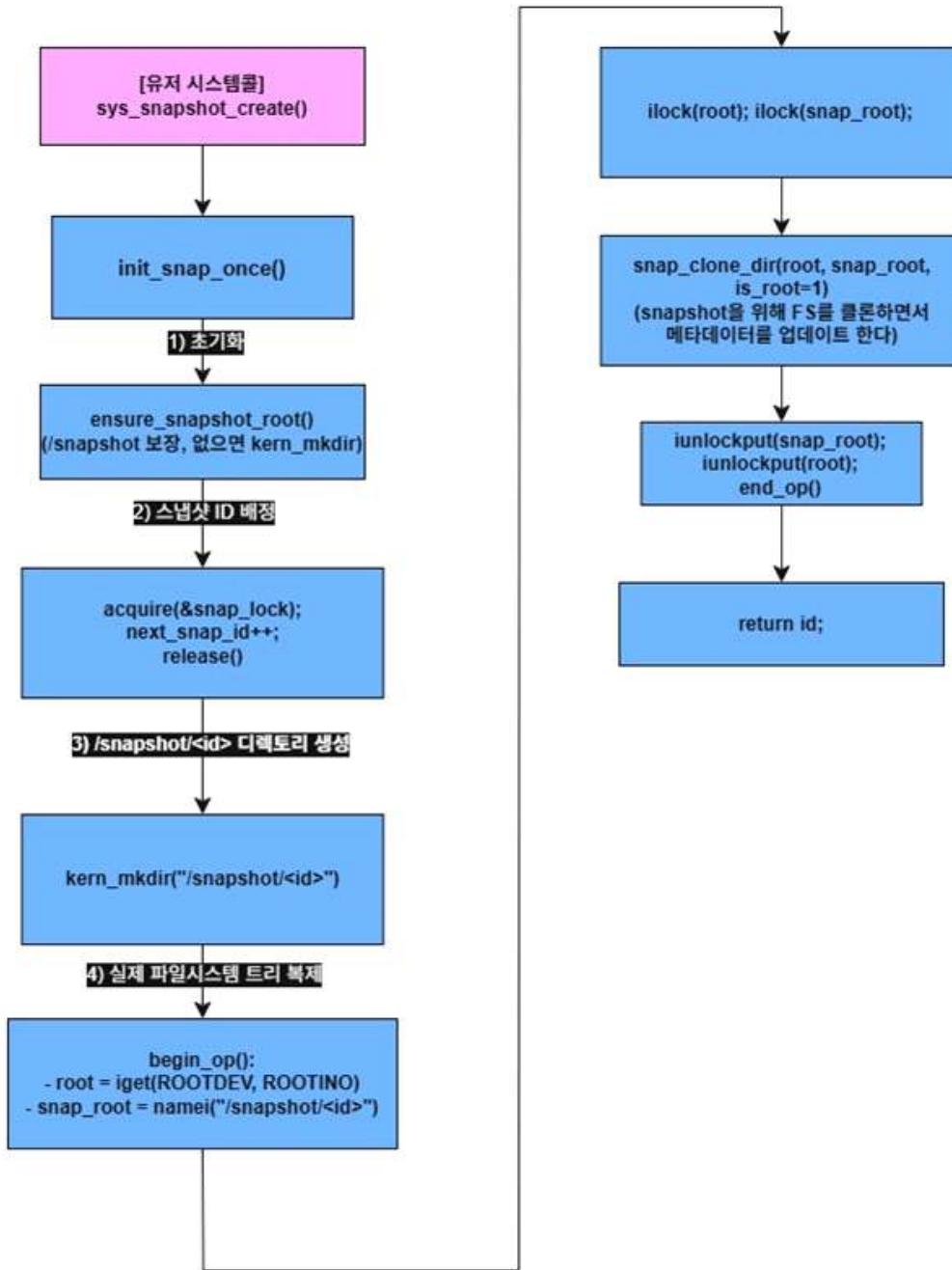
static void dir_unlink_at(struct inode *dp, uint off, struct inode *ip);

부모 디렉토리(dp)의 off 위치에 있는 dirent를 비우고,

디렉토리였다면 dp->nlink--, 자식 inode의 nlink--까지 처리하는 공용 unlink 헬퍼이다.

스냅샷 삭제, 루트 초기화, 일반 unlink 등에서 공통으로 사용된다.

1. snap_create



snap_create 흐름도 설명:

스냅샷 생성은 시스템 전체 파일시스템의 현재 상태를 /snapshot/<id> 디렉토리 트리로 캡처하는 과정이다.

먼저 init_snap_once()를 통해 스냅샷용 락과 ID 할당기를 한 번만 초기화하고, ensure_snapshot_root()로 /snapshot 디렉토리가 존재하는지 확인한 뒤 없으면 kern_mkdir("/snapshot")로 생성한다.

그 다음 snap_lock을 잡고 전역 스냅샷 ID 할당기(next_snap_id)에서 새로운 ID를 하나 배정한 후 /snapshot/<id> 디렉토리를 kern_mkdir()로 생성한다.

begin_op() 안에서 루트 디렉토리(/)의 inode와 방금 만든 /snapshot/<id>의 inode를 각각 iget()/namei()로 얻고, ilock(root), ilock(snap_root)를 통해 둘 다 락을 건 상태에서 snapshot_clone_dir(root, snap_root, 1)을 호출하여 실제 트리 복사를 수행한다.

snapshot_clone_dir() 내부에서는 원본 루트부터 하위 디렉토리까지를 재귀적으로 순회하면서,

디렉토리(T_DIR)는 create_subdir()로 대응되는 디렉토리를 만들고 재귀 복사하고,

장치 파일(T_DEV)은 스냅샷 대상에서 제외하며,

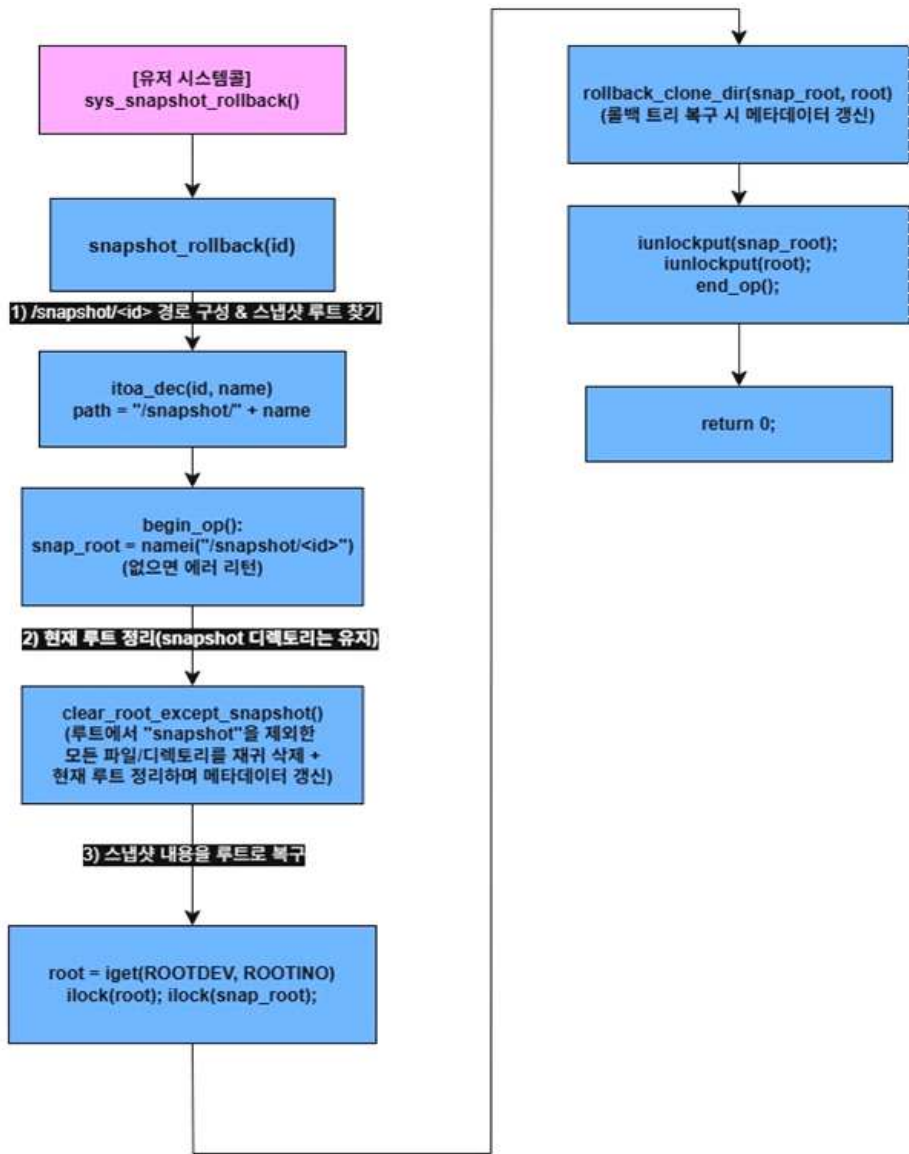
일반 파일(T_FILE)은 스냅샷 파일(T_SNAP) inode로 생성한 뒤, 원본 파일의 size와 addrs[]를 그대로 복사해 블록을 공유하게 만든다.

이때 각 스냅샷 파일에 대해 inc_snap_for_inode_blocks()를 호출하여 해당 inode가 참조하는 모든 데이터 블록 및 간접 블록에 대해 blockmeta_add_ref(b, 1)을 수행한다.

그 결과 블록마다 block_refcnt[b]++, block_snapcnt[b]++가 갱신되고, 동시에 blockmeta_write_record()를 통해 /snapshot/blockmeta 파일의 해당 레코드가 블록 단위로 즉시 갱신(in-place update) 된다.

트리 복사가 끝나면 iunlockput(snap_root), iunlockput(root)로 두 inode의 락과 참조를 해제하고 end_op()로 로그 트랜잭션을 종료한 뒤, 새로 할당된 스냅샷 ID를 리턴한다.

2. snap_rollback 흐름도



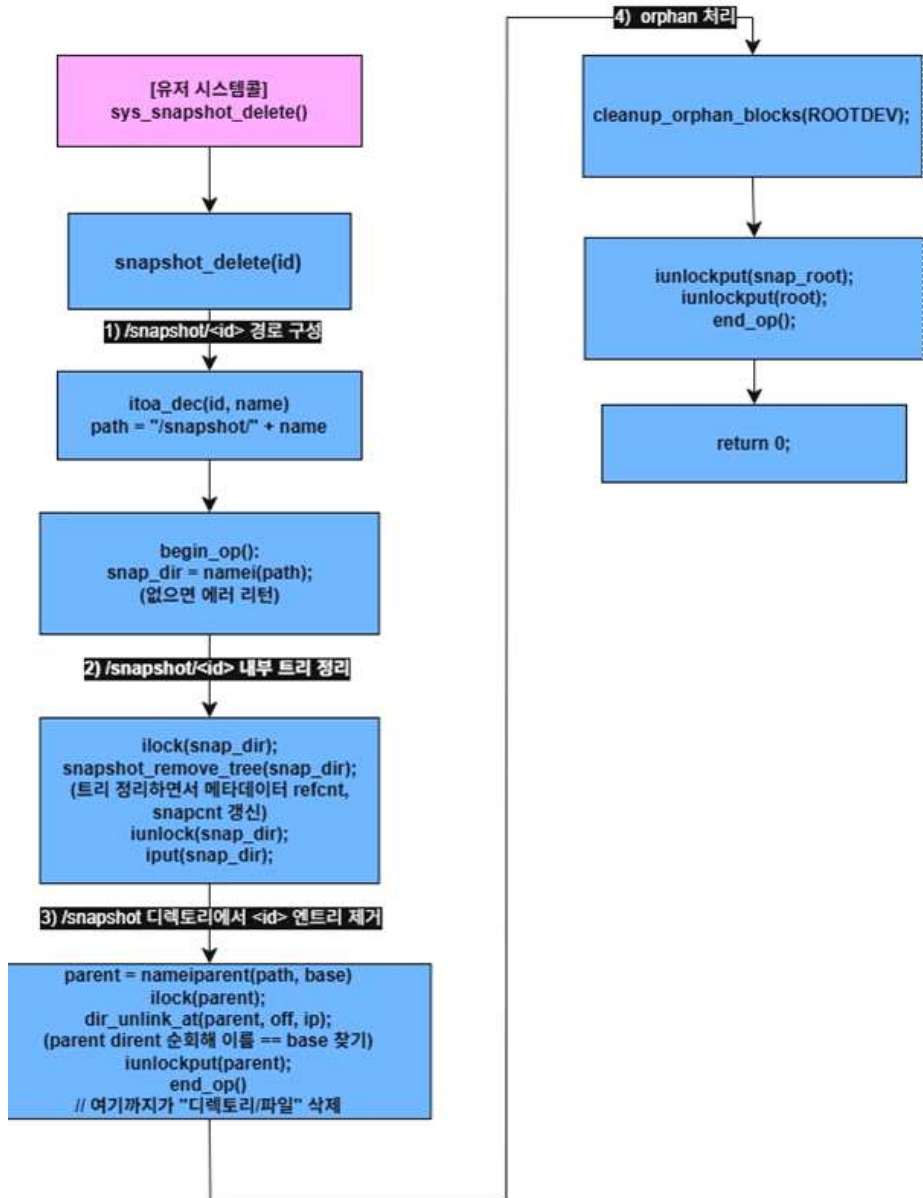
snap_rollback 흐름도 설명:

스냅샷 롤백은 /snapshot/<id>에 저장된 스냅샷 트리를 이용해, 현재 파일시스템의 내용을 해당 시점 상태로 되돌리는 기능이다. 사용자 시스템콜 sys_snapshot_rollback()은 인자로 전달된 스냅샷 ID를 받아 snapshot_rollback(id)를 호출한다. 먼저 itoa_dec()를 통해 ID를 문자열로 변환하고, "/snapshot/" + id 형태의 경로를 구성한 뒤 begin_op() 안에서 namei(path)를 호출하여 /snapshot/<id> 디렉토리 inode(snap_root)를 찾는다. 존재하지 않으면 잘못된 ID로 판단하고 에러를 반환한다. 유효한 스냅샷을 찾으면 clear_root_except_snapshot()을 호출하여 현재 루트 디렉토리(/)를 정리한다. 이 함수는 루트 아래에서 ".", "..", 그리고 "snapshot" 디렉토리를 제외한 모든 엔트리를 재귀적으로 삭제한다. 디렉토리 삭제 시에는 snapshot_remove_tree()를 통해 하위 트리를 먼저 모두 지우고, 스냅샷 파일(T_SNAP)을 삭제할 때는 dec_snap_for_inode_blocks()를 호출하여 해당 스냅샷이 참조하던 데이터 블록들에 대해 snapcnt--를 수행한다. 이 과정에서 blockmeta_dec_snap() 및 내부의 blockmeta_write_record()가 호출되어 /snapshot/blockmeta 파일의 스냅샷 카운트가 블록 단위로 즉시 갱신된다. 일반 파일/디렉토리의 블록 해제는 기존 itrunc()→bdecref() 경로를 통해 refcnt가 감소하며, 이때도 blockmeta_write_record()가 함께 호출되어 refcnt 정보가 맞게 유지된다. 루트 정리가 끝나면 root = iget(ROOTDEV, ROOTINO)로 실제 루트 inode를 가져오고, ilock(root)와 ilock(snap_root)로 두 디렉토리를 모두 잠금 상태에서 rollback_clone_dir(snap_root, root)를 호출한다. 이 함수는 스냅샷 트리(/snapshot/<id>)를 재귀적으로 순회하면서 스냅샷 디렉토리(T_DIR)는 create_subdir()를 이용해 현재 파일시스템 쪽에 일반 디렉토리(T_DIR)로 재생성하고, 스냅샷 파일(T_SNAP)은 create_file()로 일반 파일(T_FILE)을 만든 뒤, 스냅샷 inode의 size와 addrs[]를 그대로 복사해 동일한 데이터 블록을 공유하도록 만든다. 이후 inc_ref_for_inode_blocks()를 호출해 해당 T_FILE이 참조하는 모든 데이터 블록 및 간접 블록에 대해 blockmeta_add_ref_nosnap(b)를 수행함으로써 block_refcnt[b]++를 반영하고, /snapshot/blockmeta 파일의 각 레코드를 블록 단위로 in-place 업데이트한다. 이때 snapcnt는 변하지 않으므로, “스냅샷이 블록을 몇 개 참조하는지”에 대한 정보는 그대로

유지된다.

트리 복구가 끝나면 iunlockput(snap_root)와 iunlockput(root)를 통해 두 inode의 락과 참조를 해제하고, end_op()로 로그 트랜잭션을 종료한 뒤 0을 반환한다.

3. snap_delete 흐름



흐름도 설명: 스냅샷 삭제는 /snapshot/<id> 트리 전체를 제거하고, 그 결과로 더 이상 누구도 참조하지 않는 데이터 블록을 정리하여 파일시스템을 깨끗한 상태로 유지하는 과정이다. 사용자 시스템콜 sys_snapshot_delete()는 인자로 전달된 스냅샷 ID에 대해 snapshot_delete(id)를 호출한다.

먼저 itoa_dec(id, name)으로 ID를 문자열로 변환하고, "/snapshot/" + name 형태의 경로를 만들어 begin_op() 안에서 namei(path)를 호출하여 해당 스냅샷 디렉토리 inode(snap_dir)를 찾는다. 존재하지 않으면 잘못된 ID로 판단하고 오류를 반환한다.

유효한 스냅샷을 찾으면, 두 번째 단계로 /snapshot/<id> 내부 트리를 정리한다. ilock(snap_dir)를 잡은 뒤 snapshot_remove_tree(snap_dir)를 호출하면, 이 함수가 디렉토리 엔트리를 처음부터 끝까지 스캔하면서 재귀적으로 하위 트리를 모두 삭제한다.

디렉토리(T_DIR)의 경우, 먼저 자식들을 snapshot_remove_tree()로 모두 비운 다음 dir_unlink_at()을 호출하여 부모 디렉토리에서 해당 엔트리를 제거하고, 그 과정에서 부모의 nlink와 자식의 nlink가 감소한다. 자식 디렉토리 inode에 대해서는 이후 iput() 경로에서 itrunc()와 bdecref()가 호출되면서, 참조하던 데이터 블록의 refcnt가 줄고, bdecref() 안에서 blockmeta_write_record()를 통해 /snapshot/blockmeta의 해당 레코드가 즉시 in-place로 갱신된다.

스냅샷 파일(T_SNAP)의 경우, 삭제 시점에 우선 dec_snap_for_inode_blocks(child)를 호출하여 이 스냅샷 파일이 참조하던 모든 블록에 대해 snapcnt--를 수행한다. 내부적으로는 각 블록에 대해 blockmeta_dec_snap(b)가 호출되며, 이때도 blockmeta_write_record()를 통해 /snapshot/blockmeta 파일에서 해당 블록 레코드의 스냅샷 카운트가 즉시 갱신된다. 이후 dir_unlink_at()으로 디렉토리 엔트리를 지우고, iput(child) → itrunc() → bdecref() 흐름을 통해 refcnt 감소와 실제 블록 해제가 이루어진다. 이 때 refcnt 변경 역시 blockmeta_write_record()에 의해 /snapshot/blockmeta에 바로 반영된다.

/snapshot/<id> 내부 트리를 모두 제거하면, 세 번째 단계에서는 /snapshot 디렉토리에서 <id> 엔트리 자체를 지운다.

nameparent(path, base)로 부모 디렉토리(/snapshot)를 구한 뒤 ilock(parent)를 잡고, 디렉토리 엔트리들을 순회하면서 이름이 base인 항목을 찾는다. 해당 엔트리를 찾으면 ip = iget(parent->dev, de.inum)으로 자식 inode를 얻고, dir_unlink_at(parent, off, ip)를 호출하여 디렉토리 엔트리를 비우고 양쪽 nlink를 감소시킨 후, iunlock(ip); iput(ip);로 마무리한다. 이로써 /snapshot/<id> 디렉토리 자체가 디렉토리 트리에서 완전히 제거된다.

네 번째 단계에서는 end_op()로 로그 트랜잭션을 종료한 뒤, 별도의 단계로 cleanup_orphan_blocks(ROOTDEV)를 호출한다. 이 함수는 현재 메모리에 유지된 block_refcnt[] 및 block_snapcnt[] 배열과 디스크의 비트맵을 비교하면서, refcnt == 0 && snapcnt == 0이지만

비트맵상 여전히 할당(allocated)으로 표시되어 있는 “고아 데이터 블록”들을 찾아 실제 bfree()를 수행한다.

내부에서는 begin_op() / end_op()를 여러 번 나누어 사용하고, 한 트랜잭션에서 너무 많은 블록을 free하지 않도록 MAX_FREE_PER_TX 개 단위로 잘라서 처리한다.

3. 결과

(1) 기본 스냅샷 내용/구조 검증(명세 B의 “/snapshot/[ID] 캡처” 확인)

```
$ mk_test_file hi
$ ls
.          1 1 512
..         1 1 512
README    2 2 2286
cat        2 3 15664
echo       2 4 14544
forktest   2 5 8980
grep       2 6 18500
init       2 7 15168
kill       2 8 14628
ln         2 9 14524
ls         2 10 17096
mkdir      2 11 14652
rm         2 12 14632
sh         2 13 28684
stressfs   2 14 15560
usertests  2 15 63056
wc         2 16 16080
zombie     2 17 14200
print_addr 2 18 14796
snap_create 2 19 14568
snap_rollback 2 20 14808
snap_delete 2 21 14796
mk_test_file 2 22 15320
append     2 23 15120
meta_dump  2 24 15020
snapshot   1 25 48
console    3 27 0
hi         2 28 6150
$

$ print_addr hi
addr[0] : 361
addr[1] : 362
addr[2] : 363
addr[3] : 364
addr[4] : 365
addr[5] : 366
addr[6] : 367
addr[7] : 368
addr[8] : 369
addr[9] : 36a
addr[10] : 36b
addr[11] : 36c
addr[12] : 36d (INDIRECT POINTER)
addr[12] -> [0] (bn : 12) : 36e
$
```

mk_test_file hi의 결과로 hi라는 파일이 생성되었다. 해당 파일은 12개의 512B 블록+“hello\n” 등을 써서 여러 블록을 사용하는 파일 생성. 그 후 print_addr hi 명령어로 hi의 블록 주소를 확인했다.

```
$ snap_create
snapshot id = 1
$ ls /snapshot
.          1 25 64
..         1 1 512
blockmeta  2 26 2048
1          1 29 416
$
```

snap_create 명령어로 ID=1인 스냅샷을 생성했다. 이후 ls /snapshot을 통해 /snapshot/1이 생성된 것을 확인 가능하다.

```

$ ls /snapshot/1
.          1 29 416
..         1 25 64
README    4 30 2286
cat       4 31 15664
echo      4 32 14544
forktest  4 33 8980
grep      4 34 18500
init      4 35 15168
kill      4 36 14628
ln        4 37 14524
ls        4 38 17096
mkdir     4 39 14652
rm        4 40 14632
sh        4 41 28684
stressfs  4 42 15560
usertests 4 43 63056
wc        4 44 16080
zombie    4 45 14200
print_addr 4 46 14796
snap_create 4 47 14568
snap_rollback 4 48 14808
snap_delete 4 49 14796
mk_test_file 4 50 15320
append    4 51 15120
meta_dump 4 52 15020
hi        4 53 6150
$

```

그리고 /snapshot/1의 트리 구조를 확인한다. /snapshot 아래에 새로 만든 id=1인 디렉토리가 있고, blockmeta 파일이 존재한다. /snapshot/1/ 디렉토리 안에는 root(/)의 모든 엔트리가 복사되어 있어야 하지만, 다시 snapshot 디렉토리는 포함되지 않아야 한다. -> ls /snapshot/1에 snapshot이 없어야 함(명세: "/snapshot 디렉토리는 캡처하지 않음"). T_DEV 타입 파일도 존재하지 않아야 함.(소스 코드에서 T_DEV를 필터링한 상태).

(2) COW 동작+스냅샷 내용 불변성 검증(A. block COW 구현 핵심)

2-1. 스냅샷 직후의 시냅샷 파일 블록 주소 확인

```

$ print_addr /snapshot/1/hi
addr[0] : 361
addr[1] : 362
addr[2] : 363
addr[3] : 364
addr[4] : 365
addr[5] : 366
addr[6] : 367
addr[7] : 368
addr[8] : 369
addr[9] : 36a
addr[10] : 36b
addr[11] : 36c
addr[12] : 36d (INDIRECT POINTER)
addr[12] -> [0] (bn : 12) : 36e
$

$ print_addr hi
addr[0] : 361
addr[1] : 362
addr[2] : 363
addr[3] : 364
addr[4] : 365
addr[5] : 366
addr[6] : 367
addr[7] : 368
addr[8] : 369
addr[9] : 36a
addr[10] : 36b
addr[11] : 36c
addr[12] : 36d (INDIRECT POINTER)
addr[12] -> [0] (bn : 12) : 36e
$

```

스냅샷이 만든 hi의 복사본은 /snapshot/1/hi(T_SNAP 타입)이다. 이때 /snapshot/1/hi의 블록 주소들이 스냅샷 이전에 찍어둔 /hi의 블록 주소와 완전히 동일해야 한다.(실제로 완전히 동일하다) 이건 "블록 복사 없이 inode만 새로 만들고 addr[]를 공유"했다는 뜻이다.

2-2. COW가 제대로 나오는지: 원본 파일 수정


```

$ append hi XYZ
$ print_addr hi
addr[0] : 361
addr[1] : 362
addr[2] : 363
addr[3] : 364
addr[4] : 365
addr[5] : 366
addr[6] : 367
addr[7] : 368
addr[8] : 369
addr[9] : 36a
addr[10] : 36b
addr[11] : 36c
addr[12] : 371 (INDIRECT POINTER)
addr[12] -> [0] (bn : 12) : 370
$

$ print_addr /snapshot/1/hi
addr[0] : 361
addr[1] : 362
addr[2] : 363
addr[3] : 364
addr[4] : 365
addr[5] : 366
addr[6] : 367
addr[7] : 368
addr[8] : 369
addr[9] : 36a
addr[10] : 36b
addr[11] : 36c
addr[12] : 36d (INDIRECT POINTER)
addr[12] -> [0] (bn : 12) : 36e
$

```

1. append hi XYZ 이후, /snapshot/1/hi의 블록 주소는 절대 바뀌면 안된다 -> 스냅샷 내용 불변성
2. hi의 마지막 블록이나 새로 할당된 블록들은, COW 조건("스냅샷도 참조 중인 블록을 현재 FS에서 수정하려 할 때 복사")에 따라 새로운 블록 번호로 바뀌는 게 정상이다. 즉 최소한 변경이 발생한 블록(또는 추가된 블록)은 print_addr 결과에서 원래 값과 달라져야 한다. 이 테스트로 bcow가 참조 카운트(or 이 스냅샷 참조 카운트)에 따라 조건적으로 새 블록을 할당하도록 동작함을 간접 확인 가능하다. 위 결과를 보면 append hi XYZ의 결과로 끝에 추가된 데이터 부분이 할당되는 블록인 addr[12]의 주소만 변경되었다(COW)

2-3. 스냅샷 파일은 수정 불가능한지 확인

```

$ append /snapshot/1/hi XYZ
append: cannot open /snapshot/1/hi
$

```

```

370 // 2) 스냅샷(T_SNAP) 파일은 읽기 전용으로만 open 허용
371 if(ip->type == T_SNAP){
372     // 쓰기 관련 모드가 하나라도 있으면 에러
373     if(omode & (O_WRONLY | O_RDWR | O_TRUNC)){
374         iunlockput(ip);
375         end_op();
376         return -1;
377     }
378     // 여기서 omode를 강제로 O_RDONLY로 normalize
379     omode = O_RDONLY;
380 }
381 }
382

```

명세에 "snapshot 파일은 수정 불가능, 현재의 파일 시스템은 수정 가능"이라고 나와 있다. 따라서 append /snapshot/1/hi XYZ를 했을 때 명령어가 실패해야 한다. 이는 sysfile.c의 sys_open에서(오른쪽 스크린샷) 스냅샷 블록인 T_SNAP에 대해서는 읽기 전용으로만 open을 허용하도록 강제했다.

(3) snapshot_rollback이 "그 시점"으로 되돌리는지 검증

이번에는 스냅샷 다음에 원본 파일을 몇 번 수정한 후, 롤백으로 되돌릴 수 있는지 확인하였다.

3-1. 상태 변조 및 스냅샷으로 롤백

```

$ mk_test_file foo
$ append foo AAA
$ snap_rollback 1
snap_rollback: success (id=1)
$

```

foo 파일 만들었으므로 현재 디렉토리에는 foo 파일이 있었다. 그 후 foo 파일이 없는 스냅샷인 /snapshot/1로 rollback했다.

3-2. 롤백 후 파일 목록/내용 검사

```

$ snap_rollback 1
snap_rollback: success (id=1)
$ ls
.          1 1 512
..         1 1 512
README    2 2 2286
cat        2 3 15664
echo       2 4 14544
forktest   2 5 8980
grep       2 6 18500
init       2 7 15168
kill       2 8 14628
ln         2 9 14524
ls         2 10 17096
mkdir     2 11 14652
rm         2 12 14632
sh         2 13 28684
stressfs   2 14 15560
usertests  2 15 63056
wc         2 16 16080
zombie     2 17 14200
print_addr 2 18 14796
snap_create 2 19 14568
snap_rollback 2 20 14808
snap_delete 2 21 14796
mk_test_file 2 22 15320
append     2 23 15120
meta_dump  2 24 15020
snapshot   1 25 64
hi         2 28 6150
$

print_addr hi
addr[0] : 361
addr[1] : 362
addr[2] : 363
addr[3] : 364
addr[4] : 365
addr[5] : 366
addr[6] : 367
addr[7] : 368
addr[8] : 369
addr[9] : 36a
addr[10] : 36b
addr[11] : 36c
addr[12] : 36d (INDIRECT POINTER)
addr[12] -> [0] (bn : 12) : 36e
$ print_addr /snapshot/1/hi
addr[0] : 361
addr[1] : 362
addr[2] : 363
addr[3] : 364
addr[4] : 365
addr[5] : 366
addr[6] : 367
addr[7] : 368
addr[8] : 369
addr[9] : 36a
addr[10] : 36b
addr[11] : 36c
addr[12] : 36d (INDIRECT POINTER)
addr[12] -> [0] (bn : 12) : 36e
$

```

ls 명령어로 롤백된 현재 파일시스템의 트리를 보면 foo 파일이 없다. 그리고 오른쪽 사진처럼 print_addr hi와(id=1 스냅샷으로 롤백된 현재 파일시스템에서의 hi 파일의 블록 구조), print_addr /snapshot/1/hi를 실행하면 결과가 같다. 이는 snap_rollback 1이 올바르게 작동했음을 의미한다.

```

$ cat hi
0
1
2
3
4
5
6
7
8
9
0
1
hello
$

$ cat /snapshot/1/hi
0
1
2
3
4
5
6
7
8
9
0
1
hello
$

```

추가적으로 hi 파일의 내용도 /snapshot/1/hi의 내용과 같다. 즉 롤백이 성공적으로 수행되었다.

(4) snapshot_delete + 블록 해제(bfree)와 COW 상호작용 검증

여기서는 "스냅샷 삭제 후 blockmeta / 참조 관계 / 재사용"을 본다.

4-1. 새 스냅 + 수정 + 삭제까지 연속 테스트

```

$ snap_create
snapshot id = 2
$ mk_test_file bar
$ print_addr bar
addr[0] : 371
addr[1] : 372
addr[2] : 373
addr[3] : 374
addr[4] : 375
addr[5] : 376
addr[6] : 377
addr[7] : 378
addr[8] : 379
addr[9] : 37a
addr[10] : 37b
addr[11] : 37c
addr[12] : 37d (INDIRECT POINTER)
addr[12] -> [0] (bn : 12) : 37e
$ snap_create
snapshot id = 3
$
$ append bar HELLO
$ print_addr bar
addr[0] : 371
addr[1] : 372
addr[2] : 373
addr[3] : 374
addr[4] : 375
addr[5] : 376
addr[6] : 377
addr[7] : 378
addr[8] : 379
addr[9] : 37a
addr[10] : 37b
addr[11] : 37c
addr[12] : 381 (INDIRECT POINTER)
addr[12] -> [0] (bn : 12) : 380
$ print_addr /snapshot/3/bar
addr[0] : 371
addr[1] : 372
addr[2] : 373
addr[3] : 374
addr[4] : 375
addr[5] : 376
addr[6] : 377
addr[7] : 378
addr[8] : 379
addr[9] : 37a
addr[10] : 37b
addr[11] : 37c
addr[12] : 37d (INDIRECT POINTER)
addr[12] -> [0] (bn : 12) : 37e
$

```

왼쪽: snap_create로 id=2 스냅샷을 저장한 후, mk_test_file bar로 bar 파일을 만든다. print_addr bar로 bar 파일의 블록 주소를 확인한다. 그 후 snap_create로 id=3 스냅샷을 저장한다.

오른쪽: /snapshot/3이 저장된 후 append bar HELLO로 bar 파일을 수정한다. 그러면 COW가 발동하여 bar 파일의 간접 블록이 사용하는 블록 주소가 바뀌었을 것이다. 실제로 print_addr bar를 찍어보면 기존 381 -> 37d, 380->37e로 변경된 것을 확인 가능하다. 그리고 print_addr /snapshot/3/bar로 스냅샷 3에 저장된 bar 파일의 블록 주소를 보면 과거 스냅샷 캡처 시점의 블록 주소가 잘 유지되어 있다.

4-2. 스냅샷만 삭제해서 참조 카운트가 줄어드는 상황 확인

```

$ snap_delete 3
snap_delete: success (id=3)
$ ls /snapshot
.          1 25 96
..         1 1 512
blockmeta  2 26 2048
1          1 29 416
2          1 54 416
$

```

```

$ meta_dump
b 3C : ref=3 snap=2
b 3D : ref=3 snap=2
b 3E : ref=3 snap=2
b 3F : ref=3 snap=2
b 40 : ref=3 snap=2
b 41 : ref=3 snap=2

```

```

b 370 : ref=1 snap=0
b 371 : ref=1 snap=0
b 372 : ref=1 snap=0
b 373 : ref=1 snap=0
b 374 : ref=1 snap=0
b 375 : ref=1 snap=0
b 376 : ref=1 snap=0
b 377 : ref=1 snap=0
b 378 : ref=1 snap=0
b 379 : ref=1 snap=0
b 37A : ref=1 snap=0
b 37B : ref=1 snap=0
b 37C : ref=1 snap=0
b 380 : ref=1 snap=0
b 381 : ref=1 snap=0

```

왼쪽: snap_delete 3을 통해 id=3인 스냅샷을 삭제했다. 그러고 나서 ls /snapshot으로 확인해 보면 정말 id=3 스냅샷은 삭제되어있다.

오른쪽: meta_dump를 통해 메타데이터 파일에 기록된 refcnt, snapcnt 값을 확인한다. snapshot/3이 있었다면 해당 스냅샷에 파일 bar가 있으므로, 적어도 371~37C 블록까지는 ref=2, snap=1이 되었을 것이다. 하지만 왼쪽에서 snap_delete 3으로 bar 파일이 있는 스냅샷을 삭제했다. 따라서 현재 파일시스템에만 bar 파일이 남아있으므로, 371~37C 블록의 카운트는 ref=1, snap=0이 된다.

(5) 블록 재사용까지 확인(bfree가 실제로 동작했는지)

5-1. 특정 파일 + 스냅샷 조합에 대해 블록들을 완전히 날려보기 + 참조/스냅샷 카운트가 0이 되었는지 체크

```
$ mk_test_file tmp
$ print_addr tmp
addr[0] : 37d
addr[1] : 37e
addr[2] : 37f
addr[3] : 382
addr[4] : 383
addr[5] : 384
addr[6] : 385
addr[7] : 386
addr[8] : 387
addr[9] : 388
addr[10] : 389
addr[11] : 38a
addr[12] : 38b (INDIRECT POINTER)
addr[12] -> [0] (bn : 12) : 38c
$ snap_create
snapshot id = 4
$
b 37D : ref=2 snap=1
b 37E : ref=2 snap=1
b 37F : ref=2 snap=1
b 380 : ref=2 snap=1
b 381 : ref=2 snap=1
b 382 : ref=2 snap=1
b 383 : ref=2 snap=1
b 384 : ref=2 snap=1
b 385 : ref=2 snap=1
b 386 : ref=2 snap=1
b 387 : ref=2 snap=1
b 388 : ref=2 snap=1
b 389 : ref=2 snap=1
b 38A : ref=2 snap=1
b 38B : ref=2 snap=1
b 38C : ref=2 snap=1

$ rm tmp
$ meta_dump
b 37D : ref=1 snap=1
b 37E : ref=1 snap=1
b 37F : ref=1 snap=1
b 380 : ref=2 snap=1
b 381 : ref=2 snap=1
b 382 : ref=1 snap=1
b 383 : ref=1 snap=1
b 384 : ref=1 snap=1
b 385 : ref=1 snap=1
b 386 : ref=1 snap=1
b 387 : ref=1 snap=1
b 388 : ref=1 snap=1
b 389 : ref=1 snap=1
b 38A : ref=1 snap=1
b 38B : ref=1 snap=1
b 38C : ref=1 snap=1

$ snap_delete 4
snap_delete: success (id=4)
$ meta_dump
b 37C : ref=1 snap=0
b 380 : ref=1 snap=0
b 381 : ref=1 snap=0
$
```

위쪽: tmp 파일을 만들고 tmp 파일이 사용하는 블록 주소 확인, snap_create로 id=4인 스냅샷을 생성. 그 후 tmp를 삭제하고 meta_dump를 찍어본다. 그러면 ref만 하나 줄어서 ref=1, snap=1임을 확인 가능하다.

아래쪽: snap_delete 4로 tmp파일이 있는 스냅샷까지 삭제한다. 즉 tmp파일이 어디에도 존재하지 않게 된다. tmp 파일이 사용하는 블록을 완전히 삭제했다는 것은 refcnt, snapcnt를 완전히 0으로 만들었다는 뜻이다. 따라서 meta_dump 프로그램을 실행했을 때 기존에 tmp 파일이 쓰던 37d~38c 블록이 아예 나오지 않는다.(참조가 없으면 기록되지 않는다).

5-2. 실제로 블록이 재사용되는지 확인

```

$ mk_test_file new1
$ print_addr new1
addr[0] : 37d
addr[1] : 37e
addr[2] : 37f
addr[3] : 382
addr[4] : 383
addr[5] : 384
addr[6] : 385
addr[7] : 386
addr[8] : 387
addr[9] : 388
addr[10] : 389
addr[11] : 38a
addr[12] : 38b (INDIRECT POINTER)
addr[12] -> [0] (bn : 12) : 38c

```

새로 만든 파일인 new1이 사용하는 블록을 print_addr new1로 확인해 보았을 때, tmp가 쓰던 블록 번호들인 37d~38c가 다시 등장하여 쓰인다. 즉 bfree+재사용이 잘 되는 것을 확인 가능하다. 5-1, 5-2 실험을 통해 명세의 A. block COW 구현에서 "블록에 대한 참조가 모두 없어질 때 해당 블록을 할당 해제"라는 명세 문장을 만족한다고 볼 수 있다.

4. 소스코드

1. append.c

```

// append.c
// 파일 끝에 문자열을 붙이는 도구
// 사용 목적: writei()의 COW 동작 검증하기
// 스냅샷이 잡고 있는 블록을 덮어쓸 때 COW가 제대로 되는지, 간접 블록 영역을 건드릴 때 COW가 잘 되는지 검증하는 용도
#include "types.h"
#include "stat.h"
#include "user.h"
#include "fcntl.h"

```

```

int main(int argc, char *argv[])
{
    // 인자 체크
    // 사용법: append <파일명> "넣으려는 문자열" ex) append hi "hello_snapshot"
    if(argc != 3){
        printf(2, "Usage: append filename string\n");
        exit();
    }

    // open the file for read+write, create if needed
    // 파일이 존재하면 읽기+쓰기로 열고, 없으면 새로 만들고 쓰기 시작
    int fd = open(argv[1], O_RDWR | O_CREATE);
    if(fd < 0){
        printf(2, "append: cannot open %s\n", argv[1]);
        exit();
    }

    // move offset to the end by reading until EOF
    // 1바이트씩 끝까지 읽어서, 파일 오프셋을 EOF 위치로 옮긴다(lseek 대응)
    // 루프가 끝나면 fd의 현재 위치 = 파일 끝.
    char buf[1];
    while(read(fd, buf, 1) == 1) { // drain to EOF
    }
}

```

```

// now at end, write the string
// 이제 오프셋이 EOF에 있으니, 그대로 쓰면 append 효과가 있다.
if(write(fd, argv[2], strlen(argv[2])) < 0){
    printf(2, "append: write failed\n");
    close(fd);
    exit();
}
close(fd);
exit();
}

2. mk_test_file.c
// mk_test_file.c
// 테스트용으로 직접 블록 + 간접 블록까지 한번에 테스트 할 수 있는 큰 파일 만드는 사용자 프로그램
// 사용법: mk_test_file <파일명> 으로 실행하면 된다
// 생성되는 파일 구조:
// 블록0: "0 0 0 ... 0 \n"
// 블록1: "1 0 0 ... 0 \n"
// 블록11: "1 0 0 ... 0 \n"
// 그 뒤에 "hello\n"이 간접 블록으로 이어짐

#include "types.h"
#include "fcntl.h"
#include "user.h"

int main(int argc, char *argv[]){
    int fd;
    // 인자 없이 사용하면 종료
    if(argc < 2){
        printf(1, "need argv[1]\n");
        exit();
    }
    // 잘못된 open이면 종료
    if((fd = open(argv[1], O_CREATE | O_WRONLY)) < 0){
        printf(1, "open error for %s\n", argv[0]);
        exit();
    }
    char buf[513];
    // buf[0]: '0'~'9' 같은 숫자 문자(블록 번호용)로 채운다
    // buf[1..510]: 전부 0으로 채운다
    // buf[511]: '\n'
    for(int i = 1; i < 511; i++){
        buf[i] = 0;
    }
    buf[511] = '\n';

    // xv6의 NDIRECT 값은 기본적으로 12라서, 이 루프는 파일의 직접 블록 12개를 정확히 채우는 역할을 한다
    // i=0 -> '0', i=1 -> '1', ... i=10 -> '0', i=11 -> '1'
    for(int i=0; i < 12; i++){
        buf[0] = i % 10 + '0';
        write(fd, buf, 512);
    }
    // 이미 위에서 12개의 블록(12*512바이트)을 써서, 파일 오프셋은 정확히 12*512 바이트 지점이다.
    // 이제는 간접 블록 영역이 시작되는 위치이다.
    // 직접 블록이 다 채워지면 그 뒤에 hello를 쓴다. 이는 간접 블록 영역에 작성된다
    // 즉 간접 블록에도 COW가 잘 되는지까지 한번에 테스트하는 용도이다.
    char *str = "hello\n";
    write(fd, str, 6);
    close(fd);
    exit();
}

```

```
}
```

3. print_addr.c

```
//print_addr.c
```

```
// 커널 쪽 print_addr 시스템콜을 호출하는 유저 프로그램(wrapper)
```

```
// 출력 결과 해석
```

```
// addr[0...11]: 파일의 직접 블록들이 가리키는 디스크 블록 번호를 출력
```

```
// addr[12]: 간접 블록 자체의 주소 출력
```

```
// addr[12] -> [0]: 간접 블록 내의 첫 번째 엔트리 -> 논리 블록 bn = 12가 가리키는 실제 데이터 블록 번호를 출력
```

```
// 사용 목적: 특정 파일의 물리 디스크 블록 매핑이 어떻게 변하는지를 직접 눈으로 확인하는 도구이다.
```

```
#include "types.h"
```

```
#include "stat.h"
```

```
#include "user.h"
```

```
#include "fcntl.h"
```

```
int
```

```
main(int argc, char *argv[])
```

```
{
```

```
    // 인자 개수 체크
```

```
    // 사용 방법: print_addr <파일 이름>
```

```
    if(argc != 2){
```

```
        printf(1, "Usage: print_addr filename\n");
```

```
        exit();
```

```
    }
```

```
    // 존재 여부 정도만 확인
```

```
    int fd = open(argv[1], O_RDONLY);
```

```
    if(fd < 0){
```

```
        printf(1, "print_addr: cannot open %s\n", argv[1]);
```

```
        exit();
```

```
    }
```

```
    close(fd);
```

```
    // 커널의 print_addr syscall 호출 (커널에서 cprintf로 출력)
```

```
    if(print_addr(argv[1]) < 0){
```

```
        printf(1, "print_addr: syscall failed\n");
```

```
    }
```

```
    exit();
```

```
}
```

4. snap_create.c

```
// snap_create.c
```

```
// 시스템콜 snap_create()를 호출하여 스냅샷을 만드는 프로그램
```

```
// id는 increment하게 증가하게 했으며, 삭제되어 중간에 비는 id가 있더라도 무시하고 계속 증가하도록 구현함(명세 언급 X)
```

```
#include "types.h"
```

```
#include "stat.h"
```

```
#include "user.h"
```

```
int
```

```
main(int argc, char *argv[])
```

```
{
```

```
    int id = snapshot_create();
```

```
    if(id < 0){
```

```
        printf(1, "snap_create: snapshot_create failed\n");
```

```
    } else {
```

```
        printf(1, "snapshot id = %d\n", id);
```

```
    }
```

```
    exit();
```

```

}
5. snap_delete.c
// snap_delete.c
// 시스템콜 snapshot_delete()를 호출하여 인자로 받은 ID의 스냅샷 파일을 삭제하는 프로그램
#include "types.h"
#include "stat.h"
#include "user.h"

```

```

int
main(int argc, char *argv[])
{
    if(argc != 2){
        printf(1, "Usage: snap_delete id\n");
        exit();
    }

    int id = atoi(argv[1]);
    int r = snapshot_delete(id);
    if(r < 0){
        printf(1, "snap_delete: failed (id=%d)\n", id);
    } else {
        printf(1, "snap_delete: success (id=%d)\n", id);
    }

    exit();
}

```

```

6. snap_rollback.c
// snap_rollback.c
// 시스템콜 snapshot_rollback()을 호출하여 인자로 받은 ID의 스냅샷으로 현재 파일 시스템을 복구하는 프로그램
#include "types.h"
#include "stat.h"
#include "user.h"

```

```

int
main(int argc, char *argv[])
{
    if(argc != 2){
        printf(1, "Usage: snap_rollback id\n");
        exit();
    }

    // 인자로 받은 id는 char *이므로, 정수로 처리하기 위해 atoi()를 사용했다
    int id = atoi(argv[1]);
    int r = snapshot_rollback(id);
    if(r < 0){
        printf(1, "snap_rollback: failed (id=%d)\n", id);
    } else {
        printf(1, "snap_rollback: success (id=%d)\n", id);
    }

    exit();
}

```

```

7. meta_dump.c
// meta_dump.c
#include "types.h"
#include "stat.h"
#include "user.h"
#include "fcntl.h"

```

```

struct block_meta {
    uchar ref;
    uchar snap;
};

int
main(int argc, char *argv[])
{
    int fd;
    struct block_meta m;
    int idx = 0;

    fd = open("/snapshot/blockmeta", O_RDONLY);
    if(fd < 0){
        printf(1, "bmdump: cannot open /snapshot/blockmeta\n");
        exit();
    }

    // 한 레코드(struct block_meta)씩 읽으면서,
    // ref==0 && snap==0 인 블록은 그냥 건너뛰고,
    // 값이 있는 블록만 출력
    while(read(fd, &m, sizeof(m)) == sizeof(m)){
        if(m.ref != 0 || m.snap != 0){
            printf(1, "b %x : ref=%d snap=%d\n", idx, m.ref, m.snap);
        }
        idx++;
    }

    close(fd);
    exit();
}

```

8. Makefile

```

OBS = \
    bio.o\
    console.o\
    exec.o\
    file.o\
    fs.o\
    ide.o\
    ioapic.o\
    kalloc.o\
    kbd.o\
    lapic.o\
    log.o\
    main.o\
    mp.o\
    picirq.o\
    pipe.o\
    proc.o\
    sleeplock.o\
    spinlock.o\
    string.o\
    swtch.o\
    syscall.o\
    sysfile.o\
    sysproc.o\
    trapasm.o\

```



```

trap.o\
uart.o\
vectors.o\
vm.o\

# Cross-compiling (e.g., on Mac OS X)
# TOOLPREFIX = i386-jos-elf

# Using native tools (e.g., on X86 Linux)
#TOOLPREFIX =

# Try to infer the correct TOOLPREFIX if not set
ifndef TOOLPREFIX
TOOLPREFIX := $(shell if i386-jos-elf-objdump -i 2>&1 | grep '^elf32-i386$$' >/dev/null 2>&1; \
    then echo 'i386-jos-elf-'; \
    elif objdump -i 2>&1 | grep 'elf32-i386' >/dev/null 2>&1; \
    then echo ''; \
    else echo "***" 1>&2; \
    echo "*** Error: Couldn't find an i386-*-elf version of GCC/binutils." 1>&2; \
    echo "*** Is the directory with i386-jos-elf-gcc in your PATH?" 1>&2; \
    echo "*** If your i386-*-elf toolchain is installed with a command" 1>&2; \
    echo "*** prefix other than 'i386-jos-elf-', set your TOOLPREFIX" 1>&2; \
    echo "*** environment variable to that prefix and run 'make' again." 1>&2; \
    echo "*** To turn off this error, run 'gmake TOOLPREFIX= ...'." 1>&2; \
    echo "***" 1>&2; exit 1; fi)
endif

# If the makefile can't find QEMU, specify its path here
# QEMU = qemu-system-i386

# Try to infer the correct QEMU
ifndef QEMU
QEMU = $(shell if which qemu > /dev/null; \
    then echo qemu; exit; \
    elif which qemu-system-i386 > /dev/null; \
    then echo qemu-system-i386; exit; \
    elif which qemu-system-x86_64 > /dev/null; \
    then echo qemu-system-x86_64; exit; \
    else \
    qemu=/Applications/Q.app/Contents/MacOS/i386-softmmu.app/Contents/MacOS/i386-softmmu; \
    if test -x $$qemu; then echo $$qemu; exit; fi; fi; \
    echo "***" 1>&2; \
    echo "*** Error: Couldn't find a working QEMU executable." 1>&2; \
    echo "*** Is the directory containing the qemu binary in your PATH" 1>&2; \
    echo "*** or have you tried setting the QEMU variable in Makefile?" 1>&2; \
    echo "***" 1>&2; exit 1)
endif

CC = $(TOOLPREFIX)gcc
AS = $(TOOLPREFIX)gas
LD = $(TOOLPREFIX)ld
OBJCOPY = $(TOOLPREFIX)objcopy
OBJDUMP = $(TOOLPREFIX)objdump
CFLAGS = -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -Werror
        -fno-omit-frame-pointer
CFLAGS += $(shell $(CC) -fno-stack-protector -E -x c /dev/null >/dev/null 2>&1 && echo -fno-stack-protector)
ASFLAGS = -m32 -gdwarf-2 -Wa,-divide
# FreeBSD ld wants `elf_i386_fbsd'

```

```
LDFLAGS += -m $(shell $(LD) -V | grep elf_i386 2>/dev/null | head -n 1)
```

```
# Disable PIE when possible (for Ubuntu 16.10 toolchain)
ifneq ($(shell $(CC) -dumpspeaks 2>/dev/null | grep -e '^[^f]no-pie'),)
CFLAGS += -fno-pie -no-pie
endif
ifneq ($(shell $(CC) -dumpspeaks 2>/dev/null | grep -e '^[^f]nopie'),)
CFLAGS += -fno-pie -nopie
endif
```

```
xv6.img: bootblock kernel
dd if=/dev/zero of=xv6.img count=10000
dd if=bootblock of=xv6.img conv=notrunc
dd if=kernel of=xv6.img seek=1 conv=notrunc
```

```
xv6memfs.img: bootblock kernelmemfs
dd if=/dev/zero of=xv6memfs.img count=10000
dd if=bootblock of=xv6memfs.img conv=notrunc
dd if=kernelmemfs of=xv6memfs.img seek=1 conv=notrunc
```

```
bootblock: bootasm.S bootmain.c
$(CC) $(CFLAGS) -fno-pic -O -nostdinc -I. -c bootmain.c
$(CC) $(CFLAGS) -fno-pic -nostdinc -I. -c bootasm.S
$(LD) $(LDFLAGS) -N -e start -Ttext 0x7C00 -o bootblock.o bootasm.o bootmain.o
$(OBJDUMP) -S bootblock.o > bootblock.asm
$(OBJCOPY) -S -O binary -j .text bootblock.o bootblock
./sign.pl bootblock
```

```
entryother: entryother.S
$(CC) $(CFLAGS) -fno-pic -nostdinc -I. -c entryother.S
$(LD) $(LDFLAGS) -N -e start -Ttext 0x7000 -o bootblockother.o entryother.o
$(OBJCOPY) -S -O binary -j .text bootblockother.o entryother
$(OBJDUMP) -S bootblockother.o > entryother.asm
```

```
initcode: initcode.S
$(CC) $(CFLAGS) -nostdinc -I. -c initcode.S
$(LD) $(LDFLAGS) -N -e start -Ttext 0 -o initcode.out initcode.o
$(OBJCOPY) -S -O binary initcode.out initcode
$(OBJDUMP) -S initcode.o > initcode.asm
```

```
kernel: $(OBJS) entry.o entryother initcode kernel.ld
$(LD) $(LDFLAGS) -T kernel.ld -o kernel entry.o $(OBJS) -b binary initcode entryother
$(OBJDUMP) -S kernel > kernel.asm
$(OBJDUMP) -t kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > kernel.sym
```

```
# kernelmemfs is a copy of kernel that maintains the
# disk image in memory instead of writing to a disk.
# This is not so useful for testing persistent storage or
# exploring disk buffering implementations, but it is
# great for testing the kernel on real hardware without
# needing a scratch disk.
```

```
MEMFSOBJS = $(filter-out ide.o,$(OBJS)) memide.o
```

```
kernelmemfs: $(MEMFSOBJS) entry.o entryother initcode kernel.ld fs.img
$(LD) $(LDFLAGS) -T kernel.ld -o kernelmemfs entry.o $(MEMFSOBJS) -b binary initcode entryother fs.img
$(OBJDUMP) -S kernelmemfs > kernelmemfs.asm
$(OBJDUMP) -t kernelmemfs | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > kernelmemfs.sym
```

```
tags: $(OBJS) entryother.S _init
```

```

etags *.S *.c

vectors.S: vectors.pl
./vectors.pl > vectors.S

ULIB = ulib.o usys.o printf.o umalloc.o

_%: %.o $(ULIB)
$(LD) $(LDFLAGS) -N -e main -Ttext 0 -o $@ $^
$(OBJDUMP) -S $@ > $.asm
$(OBJDUMP) -t $@ | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > $.sym

_forktest: forktest.o $(ULIB)
# forktest has less library code linked in - needs to be small
# in order to be able to max out the proc table.
$(LD) $(LDFLAGS) -N -e main -Ttext 0 -o _forktest forktest.o ulib.o usys.o
$(OBJDUMP) -S _forktest > forktest.asm

mkfs: mkfs.c fs.h
gcc -Werror -Wall -o mkfs mkfs.c

# Prevent deletion of intermediate files, e.g. cat.o, after first build, so
# that disk image changes after first build are persistent until clean. More
# details:
# http://www.gnu.org/software/make/manual/html_node/Chained-Rules.html
.PRECIOUS: %.o

UPROGS=\
_cat\
_echo\
_forktest\
_grep\
_init\
_kill\
_ln\
_ls\
_mkdir\
_rm\
_sh\
_stressfs\
_usertests\
_wc\
_zombie\
_print_addr\
_snap_create\
_snap_rollback\
_snap_delete\
_mk_test_file\
_append\
_meta_dump\

fs.img: mkfs README $(UPROGS)
./mkfs fs.img README $(UPROGS)

-include *.d

clean:
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \

```

```

*.o *.d *.asm *.sym vectors.S bootblock entryother \
initcode initcode.out kernel xv6.img fs.img kernelmemfs \
xv6memfs.img mkfs .gdbinit \
$(UPROGS)

# make a printout
FILES = $(shell grep -v '^\\#' runoff.list)
PRINT = runoff.list runoff.spec README toc.hdr toc.ftr $(FILES)

xv6.pdf: $(PRINT)
    ./runoff
    ls -l xv6.pdf

print: xv6.pdf

# run in emulators

bochs : fs.img xv6.img
    if [ ! -e .bochsrc ]; then ln -s dot-bochsrc .bochsrc; fi
    bochs -q

# try to generate a unique GDB port
GDBPORT = $(shell expr `id -u` % 5000 + 25000)
# QEMU's gdb stub command line changed in 0.11
QEMUGDB = $(shell if $(QEMU) -help | grep -q '^~gdb'; \
    then echo "-gdb tcp::$(GDBPORT)"; \
    else echo "-s -p $(GDBPORT)"; fi)

ifndef CPUS
CPUS := 2
endif
QEMUOPTS = -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img,index=0,media=disk,format=raw -smp
$(CPUS) -m 512 $(QEMUEXTRA)

qemu: fs.img xv6.img
    $(QEMU) -serial mon:stdio $(QEMUOPTS)

qemu-memfs: xv6memfs.img
    $(QEMU) -drive file=xv6memfs.img,index=0,media=disk,format=raw -smp $(CPUS) -m 256

qemu-nox: fs.img xv6.img
    $(QEMU) -nographic $(QEMUOPTS)

.gdbinit: .gdbinit.tmpl
    sed "s/localhost:1234/localhost:$(GDBPORT)/" < $^ > $@

qemu-gdb: fs.img xv6.img .gdbinit
    @echo "*** Now run 'gdb'." 1>&2
    $(QEMU) -serial mon:stdio $(QEMUOPTS) -S $(QEMUGDB)

qemu-nox-gdb: fs.img xv6.img .gdbinit
    @echo "*** Now run 'gdb'." 1>&2
    $(QEMU) -nographic $(QEMUOPTS) -S $(QEMUGDB)

# CUT HERE

# prepare dist for students
# after running make dist, probably want to
# rename it to rev0 or rev1 or so on and then
# check in that version.

```

```

EXTRA=\
mkfs.c ulib.c user.h cat.c echo.c forktest.c grep.c kill.c\
ln.c ls.c mkdir.c rm.c stressfs.c usertests.c wc.c zombie.c\
printf.c umalloc.c\
README dot-bochsrc *.pl toc.* runoff runoff1 runoff.list\
.gdbinit.tmpl gdbutil\

```

```

dist:
rm -rf dist
mkdir dist
for i in $(FILES); \
do \
    grep -v PAGEBREAK $$i >dist/$$i; \
done
sed '/CUT HERE/,,$$d' Makefile >dist/Makefile
echo >dist/runoff.spec
cp $(EXTRA) dist

```

```

dist-test:
rm -rf dist
make dist
rm -rf dist-test
mkdir dist-test
cp dist/* dist-test
cd dist-test; $(MAKE) print
cd dist-test; $(MAKE) bochs || true
cd dist-test; $(MAKE) qemu

```

```

# update this rule (change rev#) when it is time to
# make a new revision.

```

```

tar:
rm -rf /tmp/xv6
mkdir -p /tmp/xv6
cp dist/* dist/.gdbinit.tmpl /tmp/xv6
(cd /tmp; tar cf - xv6) | gzip >xv6-rev10.tar.gz # the next one will be 10 (9/17)

```

```

.PHONY: dist-test dist

```

9. defs.h

```

struct buf;
struct context;
struct file;
struct inode;
struct pipe;
struct proc;
struct rtcdate;
struct spinlock;
struct sleeplock;
struct stat;
struct superblock;

```

```

// bio.c
void          binit(void);
struct buf*   bread(uint, uint);
void          brelse(struct buf*);
void          bwrite(struct buf*);

```

```

// console.c
void      consoleinit(void);
void      cprintf(char*, ...);
void      consoleintr(int (*)(void));
void      panic(char*) __attribute__((noreturn));

// exec.c
int       exec(char*, char**);

// file.c
struct file* filealloc(void);
void      fileclose(struct file*);
struct file* filedup(struct file*);
void      fileinit(void);
int       fileread(struct file*, char*, int n);
int       filestat(struct file*, struct stat*);
int       filewrite(struct file*, char*, int n);

// fs.c
void      readsb(int dev, struct superblock *sb);
int       dirlink(struct inode*, char*, uint);
struct inode* dirlookup(struct inode*, char*, uint*);
struct inode* ialloc(uint, short);
struct inode* idup(struct inode*);
void      iinit(int dev);
void      ilock(struct inode*);
void      iput(struct inode*);
void      iunlock(struct inode*);
void      iunlockput(struct inode*);
void      iupdate(struct inode*);
int       namecmp(const char*, const char*);
struct inode* namei(char*);
struct inode* nameiparent(char*, char*);
int       readi(struct inode*, char*, uint, uint);
void      stati(struct inode*, struct stat*);
int       writei(struct inode*, char*, uint, uint);

// ide.c
void      ideinit(void);
void      ideintr(void);
void      iderw(struct buf*);

// ioapic.c
void      ioapicenable(int irq, int cpu);
extern uchar ioapicid;
void      ioapicinit(void);

// kalloc.c
char*     kalloc(void);
void      kfree(char*);
void      kinit1(void*, void*);
void      kinit2(void*, void*);

// kbd.c
void      kbdtintr(void);

// lapic.c
void      cmostime(struct rtcdate *r);

```

```

int          lapicid(void);
extern volatile uint*    lapic;
void         lapiceoi(void);
void         lapicinit(void);
void         lapicstartap(uchar, uint);
void         microdelay(int);

// log.c
void         initlog(int dev);
void         log_write(struct buf*);
void         begin_op();
void         end_op();

// mp.c
extern int    ismp;
void         mpinit(void);

// picirq.c
void         picenable(int);
void         picinit(void);

// pipe.c
int          pipealloc(struct file**, struct file**);
void         pipeclose(struct pipe*, int);
int          piperead(struct pipe*, char*, int);
int          pipewrite(struct pipe*, char*, int);

//PAGEBREAK: 16
// proc.c
int          cpuid(void);
void         exit(void);
int          fork(void);
int          growproc(int);
int          kill(int);
struct cpu*  mycpu(void);
struct proc* myproc();
void         pinit(void);
void         procdump(void);
void         scheduler(void) __attribute__((noreturn));
void         sched(void);
void         setproc(struct proc*);
void         sleep(void*, struct spinlock*);
void         userinit(void);
int          wait(void);
void         wakeup(void*);
void         yield(void);

// swtch.S
void         swtch(struct context**, struct context*);

// spinlock.c
void         acquire(struct spinlock*);
void         getcallerpcs(void*, uint*);
int          holding(struct spinlock*);
void         initlock(struct spinlock*, char*);
void         release(struct spinlock*);
void         pushcli(void);
void         popcli(void);

```



```

// sleeplock.c
void      acquiresleep(struct sleeplock*);
void      releasesleep(struct sleeplock*);
int       holdingsleep(struct sleeplock*);
void      initsleeplock(struct sleeplock*, char*);

// string.c
int       memcmp(const void*, const void*, uint);
void*     memmove(void*, const void*, uint);
void*     memset(void*, int, uint);
char*     safestrcpy(char*, const char*, int);
int       strlen(const char*);
int       strncmp(const char*, const char*, uint);
char*     strncpy(char*, const char*, int);

// syscall.c
int       argint(int, int*);
int       argptr(int, char**, int);
int       argstr(int, char**);
int       fetchint(uint, int*);
int       fetchstr(uint, char**);
void      syscall(void);

// timer.c
void      timerinit(void);

// trap.c
void      idtinit(void);
extern uint ticks;
void      tvinit(void);
extern struct spinlock tickslock;

// uart.c
void      uartinit(void);
void      uartintr(void);
void      uartputc(int);

// vm.c
void      seginit(void);
void      kvmalloc(void);
pde_t*    setupkvm(void);
char*     uva2ka(pde_t*, char*);
int       allocuvmm(pde_t*, uint, uint);
int       deallocuvmm(pde_t*, uint, uint);
void      freevm(pde_t*);
void      inituvm(pde_t*, char*, uint);
int       loaduvm(pde_t*, char*, struct inode*, uint, uint);
pde_t*    copyuvm(pde_t*, uint);
void      switchuvm(struct proc*);
void      switchkvm(void);
int       copyout(pde_t*, uint, void*, uint);
void      clearpteu(pde_t *pgdir, char *uva);

// number of elements in fixed-size array
#define NELEM(x) (sizeof(x)/sizeof((x)[0]))

int snapshot_create(void);

```

```

int snapshot_rollback(int);
int snapshot_delete(int);

// 시스템콜 래퍼
int sys_snapshot_create(void);
int sys_snapshot_rollback(void);
int sys_snapshot_delete(void);
10. fs.c
// File system implementation. Five layers:
//   + Blocks: allocator for raw disk blocks.
//   + Log: crash recovery for multi-step updates.
//   + Files: inode allocator, reading, writing, metadata.
//   + Directories: inode with special contents (list of other inodes!)
//   + Names: paths like /usr/rtn/xv6/fs.c for convenient naming.
//
// This file contains the low-level file system manipulation
// routines. The (higher-level) system call implementations
// are in sysfile.c.

#include "types.h"
#include "defs.h"
#include "param.h"
#include "stat.h"
#include "mmu.h"
#include "proc.h"
#include "spinlock.h"
#include "sleeplock.h"
#include "fs.h"
#include "buf.h"
#include "file.h"

// [B: 스냅샷/정리] LOG 트랜잭션 하나에서 free 할 수 있는 최대 블록 수 제한
// cleanup_orphan_blocks()에서 여러 번 begin_op/end_op로 나누기 위해 사용
// LOGSIZE는 param.c에 정의되어 있음
#define MAX_FREE_PER_TX (LOGSIZE / 2)

#define BLOCKMETA_CHUNK 16 // 한 트랜잭션에서 쓸 최대 레코드 수

// [편의 매크로] 기존 xv6에는 없던 간단한 min 매크로
#define min(a, b) ((a) < (b) ? (a) : (b))

void itrunc(struct inode*);
// there should be one superblock per disk device, but we run with
// only one device

struct inode* iget(uint dev, uint inum);

void rebuild_block_meta(int dev); // 부팅 시 block_refcnt/block_snapcnt 재구축
static int need_cow(uint b); // 특정 블록이 스냅샷과 공유 중인지 확인
static void set_block(struct inode *ip, uint bn, uint newb); // inode의 bn번째 논리 블록이 어떤 디스크 블록을 가리킬지 갱신
static void read_dinode_nolock(int dev, uint inum, struct dinode *out); // 락 없이 디스크에서 dinode 한 개 읽기(rebuild_block_meta용)

// 참조는 없는데 비트맵 상 할당된 블록을 제거하는 함수
void cleanup_orphan_blocks(int dev);

```

```

// /snapshot/blockmeta 파일의 inode 캐시
// (처음 필요할 때 namei("/snapshot/blockmeta")로 한 번만 찾아서 보관)
static struct inode *blockmeta_ip = 0;

// superblock
struct superblock sb;

// 블록 메타데이터(refcount + snapshot count), FFSIZE는 이미 param.h에 정의돼 있음
int block_refcnt[FFSIZE]; // 각 블록을 몇 개의 inode가 사용하는지
int block_snapcnt[FFSIZE]; // 각 블록을 "스냅샷 파일"이 몇 개나 참조하는지
struct spinlock block_meta_lock;

// 메타데이터 파일 관련 헬퍼 함수들
static void blockmeta_init(int dev);
//static void blockmeta_full_sync(void);
static void blockmeta_write_record(uint b, int ref, int snap);

// Read the super block.
void
readsb(int dev, struct superblock *sb)
{
    struct buf *bp;

    bp = bread(dev, 1);
    memmove(sb, bp->data, sizeof(*sb));
    brelse(bp);
}

// Zero a block.
static void
bzero(int dev, int bno)
{
    struct buf *bp;

    bp = bread(dev, bno);
    memset(bp->data, 0, BSIZE);
    log_write(bp);
    brelse(bp);
}

// Blocks.

// Allocate a zeroed disk block.
// 기존 balloc에 block_refcnt/block_snapcnt 초기화 추가
// "새로 할당된 블록은 refcnt=1, snapcnt=0"으로 시작
// 이후 COW에서 old 블록 bdecreef, 새 블록 balloc(1) 패턴으로 사용
static uint
balloc(uint dev)
{
    int b, bi, m;
    struct buf *bp;

    bp = 0;
    for(b = 0; b < sb.size; b += BPB){
        bp = bread(dev, BBLOCK(b, sb));

```

```

for(bi = 0; bi < BPB && b + bi < sb.size; bi++){
    m = 1 << (bi % 8);
    if((bp->data[bi/8] & m) == 0){ // Is block free?
        bp->data[bi/8] |= m; // Mark block in use.
        log_write(bp);
        brelse(bp);

        uint bno = b + bi;
        bzero(dev, bno); // 새 블록 0으로 채우기

        int ref, snap;

        acquire(&block_meta_lock);
        // free 블록이면 refcnt/snapcnt가 0이어야 함을 검증
        if(block_refcnt[bno] != 0 || block_snapcnt[bno] != 0){
            release(&block_meta_lock);
            panic("balloc: stale meta for free block");
        }

        // 새 블록은 현재 파일이 단독 소유한다고 보고 refcnt=1로 시작
        block_refcnt[bno] = 1;
        block_snapcnt[bno] = 0;
        ref = block_refcnt[bno];
        snap = block_snapcnt[bno];
        release(&block_meta_lock);

        // on-disk metadata in-place update
        blockmeta_write_record(bno, ref, snap);

        return bno;
    }
}
brelse(bp);
}
panic("balloc: out of blocks");
}

// Free a disk block.
// bfree를 "참조계수 기반 해제"로 바꿈
// "비트맵만" 조작하는 원래 스타일의 bfree를 bfree_raw로 분리
// 실제 해제는 refcnt/snapcnt가 0일 때 bdecref에서 호출
static void
bfree_raw(int dev, uint b)
{
    struct buf *bp;
    int bi, m;

    bp = bread(dev, BBLOCK(b, sb));
    bi = b % BPB;
    m = 1 << (bi % 8);
    if((bp->data[bi/8] & m) == 0)
        panic("freeing free block");
    bp->data[bi/8] &= ~m;
    log_write(bp);
    brelse(bp);
}

// block_refcnt[b]-- 후 ref==0 && snap==0인 경우에만 실제로 bfree_raw()
// 스냅샷이 공유 중인 블록은 snapcnt>0이므로 실제 free되지 않음.

```

```

static void
bdecref(int dev, uint b)
{
    int ref, snap;

    acquire(&block_meta_lock);
    if(block_refcnt[b] <= 0)
        panic("bdecref: ref underflow");
    block_refcnt[b]--;
    ref = block_refcnt[b];
    snap = block_snapcnt[b];
    release(&block_meta_lock);

    // on-disk metadata in-place update
    // 메타데이터 업데이트
    blockmeta_write_record(b, ref, snap);

    if(ref == 0 && snap == 0){
        bfree_raw(dev, b);
    }
}

// Inodes.
//
// An inode describes a single unnamed file.
// The inode disk structure holds metadata: the file's type,
// its size, the number of links referring to it, and the
// list of blocks holding the file's content.
//
// The inodes are laid out sequentially on disk at
// sb.startinode. Each inode has a number, indicating its
// position on the disk.
//
// The kernel keeps a cache of in-use inodes in memory
// to provide a place for synchronizing access
// to inodes used by multiple processes. The cached
// inodes include book-keeping information that is
// not stored on disk: ip->ref and ip->valid.
//
// An inode and its in-memory representation go through a
// sequence of states before they can be used by the
// rest of the file system code.
//
// * Allocation: an inode is allocated if its type (on disk)
// is non-zero. ialloc() allocates, and iput() frees if
// the reference and link counts have fallen to zero.
//
// * Referencing in cache: an entry in the inode cache
// is free if ip->ref is zero. Otherwise ip->ref tracks
// the number of in-memory pointers to the entry (open
// files and current directories). iget() finds or
// creates a cache entry and increments its ref; iput()
// decrements ref.
//
// * Valid: the information (type, size, &c) in an inode
// cache entry is only correct when ip->valid is 1.
// ilock() reads the inode from

```

```

// the disk and sets ip->valid, while iput() clears
// ip->valid if ip->ref has fallen to zero.
//
// * Locked: file system code may only examine and modify
// the information in an inode and its content if it
// has first locked the inode.
//
// Thus a typical sequence is:
// ip = iget(dev, inum)
// ilock(ip)
// ... examine and modify ip->xxx ...
// iunlock(ip)
// iput(ip)
//
// ilock() is separate from iget() so that system calls can
// get a long-term reference to an inode (as for an open file)
// and only lock it for short periods (e.g., in read()).
// The separation also helps avoid deadlock and races during
// pathname lookup. iget() increments ip->ref so that the inode
// stays cached and pointers to it remain valid.
//
// Many internal file system functions expect the caller to
// have locked the inodes involved; this lets callers create
// multi-step atomic operations.
//
// The icache.lock spin-lock protects the allocation of icache
// entries. Since ip->ref indicates whether an entry is free,
// and ip->dev and ip->inum indicate which i-node an entry
// holds, one must hold icache.lock while using any of those fields.
//
// An ip->lock sleep-lock protects all ip-> fields other than ref,
// dev, and inum. One must hold ip->lock in order to
// read or write that inode's ip->valid, ip->size, ip->type, &c.

```

```

struct {
    struct spinlock lock;
    struct inode inode[NINODE];
} icache;

```

```

void
iinit(int dev)
{
    int i = 0;

    initlock(&icache.lock, "icache");
    for(i = 0; i < NINODE; i++) {
        initsleeplock(&icache.inode[i].lock, "inode");
    }

    readsb(dev, &sb);
    cprintf("sb: size %d nblocks %d ninodes %d nlog %d logstart %d\
inodestart %d bmap start %d\n", sb.size, sb.nblocks,
        sb.ninodes, sb.nlog, sb.logstart, sb.inodestart,
        sb.bmapstart);

```

```

// 블록 메타데이터 락 초기화 + refcnt/snapcnt 재구축
// 부팅 시 디스크상의 inode들을 모두 스캔하여
// block_refcnt / block_snapcnt를 다시 계산한다

```

```

initlock(&block_meta_lock, "block_meta");
rebuild_block_meta(dev);

blockmeta_init(dev);          // /snapshot/blockmeta inode 잡고 full sync
}

// 모든 inode를 스캔해서(direct/indirect 블록)
// - block_refcnt[b] : 해당 블록을 참조하는 inode 개수
// - block_snapcnt[b]: 해당 블록을 참조하는 T_SNAP inode 개수
// 를 재구성하는 함수
void
rebuild_block_meta(int dev)
{
    int i, j;
    uint inum;
    struct dinode di;
    struct buf *bp;
    uint *a;

    acquire(&block_meta_lock);
    for(i = 0; i < FSSIZE; i++){
        block_refcnt[i] = 0;
        block_snapcnt[i] = 0;
    }
    release(&block_meta_lock);

    for(inum = 1; inum < sb.ninodes; inum++){
        read_dinode_nolock(dev, inum, &di);
        if(di.type == 0)
            continue;

        // 1) refcnt: 모든 (파일/디렉토리/스냅샷) inode에 대해
        // 2) snapcnt: T_SNAP일 때만 추가로 ++
        int is_snap = (di.type == T_SNAP);

        // direct
        for(i = 0; i < NDIRECT; i++){
            uint b = di.addrs[i];
            if(b){
                acquire(&block_meta_lock);
                block_refcnt[b]++;          // 항상 증가
                if(is_snap) block_snapcnt[b]++; // 스냅샷 파일이면 snapcnt도 증가
                release(&block_meta_lock);
            }
        }

        // indirect
        if(di.addrs[NDIRECT]){
            uint ib = di.addrs[NDIRECT];

            acquire(&block_meta_lock);
            block_refcnt[ib]++;

            if(is_snap)
                block_snapcnt[ib]++;
            release(&block_meta_lock);
        }
    }
}

```

```

    bp = bread(dev, ib);
    a = (uint*)bp->data;
    for(j = 0; j < NINDIRECT; j++){
        uint b = a[j];
        if(b){
            acquire(&block_meta_lock);
            block_refcnt[b]++;
            if(is_snap) block_snapcnt[b]++;
            release(&block_meta_lock);
        }
    }
    brelse(bp);
}
}
}

```

```

//PAGEBREAK!
// Allocate an inode on device dev.
// Mark it as allocated by giving it type type.
// Returns an unlocked but allocated and referenced inode.

```

```

struct inode*
ialloc(uint dev, short type)
{
    int inum;
    struct buf *bp;
    struct dinode *dip;

    for(inum = 1; inum < sb.ninodes; inum++){
        bp = bread(dev, IBLOCK(inum, sb));
        dip = (struct dinode*)bp->data + inum%IPB;
        if(dip->type == 0){ // a free inode
            memset(dip, 0, sizeof(*dip));
            dip->type = type;
            log_write(bp); // mark it allocated on the disk
            brelse(bp);
            return iget(dev, inum);
        }
        brelse(bp);
    }
    panic("ialloc: no inodes");
}

```

```

// Copy a modified in-memory inode to disk.
// Must be called after every change to an ip->xxx field
// that lives on disk, since i-node cache is write-through.
// Caller must hold ip->lock.

```

```

void
iupdate(struct inode *ip)
{
    struct buf *bp;
    struct dinode *dip;

    bp = bread(ip->dev, IBLOCK(ip->inum, sb));
    dip = (struct dinode*)bp->data + ip->inum%IPB;
    dip->type = ip->type;
    dip->major = ip->major;
    dip->minor = ip->minor;
}

```



```

dip->nlink = ip->nlink;
dip->size = ip->size;
memmove(dip->addrs, ip->addrs, sizeof(ip->addrs));
log_write(bp);
brelse(bp);
}

// Find the inode with number inum on device dev
// and return the in-memory copy. Does not lock
// the inode and does not read it from disk.
struct inode*
iget(uint dev, uint inum)
{
    struct inode *ip, *empty;

    acquire(&icache.lock);

    // Is the inode already cached?
    empty = 0;
    for(ip = &icache.inode[0]; ip < &icache.inode[NINODE]; ip++){
        if(ip->ref > 0 && ip->dev == dev && ip->inum == inum){
            ip->ref++;
            release(&icache.lock);
            return ip;
        }
        if(empty == 0 && ip->ref == 0)    // Remember empty slot.
            empty = ip;
    }

    // Recycle an inode cache entry.
    if(empty == 0)
        panic("iget: no inodes");

    ip = empty;
    ip->dev = dev;
    ip->inum = inum;
    ip->ref = 1;
    ip->valid = 0;
    release(&icache.lock);

    return ip;
}

// Increment reference count for ip.
// Returns ip to enable ip = idup(ip1) idiom.
struct inode*
idup(struct inode *ip)
{
    acquire(&icache.lock);
    ip->ref++;
    release(&icache.lock);
    return ip;
}

// Lock the given inode.
// Reads the inode from disk if necessary.
void
ilock(struct inode *ip)

```

```

{
    struct buf *bp;
    struct dinode *dip;

    if(ip == 0 || ip->ref < 1)
        panic("ilock");

    acquiresleep(&ip->lock);

    if(ip->valid == 0){
        bp = bread(ip->dev, IBLOCK(ip->inum, sb));
        dip = (struct dinode*)bp->data + ip->inum%IPB;
        ip->type = dip->type;
        ip->major = dip->major;
        ip->minor = dip->minor;
        ip->nlink = dip->nlink;
        ip->size = dip->size;
        memmove(ip->addrs, dip->addrs, sizeof(ip->addrs));
        brelse(bp);
        ip->valid = 1;
        if(ip->type == 0)
            panic("ilock: no type");
    }
}

// Unlock the given inode.
void
iunlock(struct inode *ip)
{
    if(ip == 0 || !holdingsleep(&ip->lock) || ip->ref < 1)
        panic("iunlock");

    releasesleep(&ip->lock);
}

// Drop a reference to an in-memory inode.
// If that was the last reference, the inode cache entry can
// be recycled.
// If that was the last reference and the inode has no links
// to it, free the inode (and its content) on disk.
// All calls to iput() must be inside a transaction in
// case it has to free the inode.
void
iput(struct inode *ip)
{
    acquiresleep(&ip->lock);
    if(ip->valid && ip->nlink == 0){
        acquire(&icache.lock);
        int r = ip->ref;
        release(&icache.lock);
        if(r == 1){
            // inode has no links and no other references: truncate and free.
            itrunc(ip);
            ip->type = 0;
            iupdate(ip);
            ip->valid = 0;
        }
    }
}

```

```

releasesleep(&ip->lock);

acquire(&icache.lock);
ip->ref--;
release(&icache.lock);
}

// Common idiom: unlock, then put.
void
iunlockput(struct inode *ip)
{
    iunlock(ip);
    iput(ip);
}

//PAGEBREAK!
// Inode content
//
// The content (data) associated with each inode is stored
// in blocks on the disk. The first NDIRECT block numbers
// are listed in ip->addrs[]. The next NINDIRECT blocks are
// listed in block ip->addrs[NDIRECT].

// Return the disk block address of the nth block in inode ip.
// If there is no such block, bmap allocates one.
static uint
bmap(struct inode *ip, uint bn)
{
    uint addr, *a;
    struct buf *bp;

    if(bn < NDIRECT){
        if((addr = ip->addrs[bn]) == 0)
            ip->addrs[bn] = addr = balloc(ip->dev);
        return addr;
    }
    bn -= NDIRECT;

    if(bn < NINDIRECT){
        // Load indirect block, allocating if necessary.
        if((addr = ip->addrs[NDIRECT]) == 0)
            ip->addrs[NDIRECT] = addr = balloc(ip->dev);
        bp = bread(ip->dev, addr);
        a = (uint*)bp->data;
        if((addr = a[bn]) == 0){
            a[bn] = addr = balloc(ip->dev);
            log_write(bp);
        }
        brelse(bp);
        return addr;
    }

    panic("bmap: out of range");
}

// Truncate inode (discard contents).
// Only called when the inode has no links
// to it (no directory entries referring to it)

```

```

// and has no in-memory reference to it (is
// not an open file or current directory).
void
itrunc(struct inode *ip)
{
    int i, j;
    struct buf *bp;
    uint *a;

    for(i = 0; i < NDIRECT; i++){
        if(ip->addrs[i]){
            // 원래 bfree() 호출 -> bdecref()로 교체
            // 스냅샷에서 공유 중인 블록은 snapcnt>0이라 실제 free되지 않음
            bdecref(ip->dev, ip->addrs[i]);
            ip->addrs[i] = 0;
        }
    }

    if(ip->addrs[NDIRECT]){
        bp = bread(ip->dev, ip->addrs[NDIRECT]);
        a = (uint*)bp->data;
        for(j = 0; j < NINDIRECT; j++){
            if(a[j])
                // 간접 블록 엔트리도 참조계수 기반 free
                bdecref(ip->dev, a[j]);
        }
        brelse(bp);
        // 간접 블록 자체도 bdecref로 ref/snap 기반 free
        bdecref(ip->dev, ip->addrs[NDIRECT]);
        ip->addrs[NDIRECT] = 0;
    }

    ip->size = 0;
    iupdate(ip);
}

// Copy stat information from inode.
// Caller must hold ip->lock.
void
stati(struct inode *ip, struct stat *st)
{
    st->dev = ip->dev;
    st->ino = ip->inum;
    st->type = ip->type;
    st->nlink = ip->nlink;
    st->size = ip->size;
}

//PAGEBREAK!
// Read data from inode.
// Caller must hold ip->lock.
int
readi(struct inode *ip, char *dst, uint off, uint n)
{
    uint tot, m;
    struct buf *bp;

    if(ip->type == T_DEV){

```

```

    if(ip->major < 0 || ip->major >= NDEV || !devsw[ip->major].read)
        return -1;
    return devsw[ip->major].read(ip, dst, n);
}

if(off > ip->size || off + n < off)
    return -1;
if(off + n > ip->size)
    n = ip->size - off;

for(tot=0; tot<n; tot+=m, off+=m, dst+=m){
    bp = bread(ip->dev, bmap(ip, off/BSIZE));
    m = min(n - tot, BSIZE - off%BSIZE);
    memmove(dst, bp->data + off%BSIZE, m);
    brelse(bp);
}
return n;
}

// PAGEBREAK!
// Write data to inode.
// Caller must hold ip->lock.
// 파일에 데이터를 쓰는 함수(스냅샷 로직 추가)
// 요약: 버퍼(src)의 n 바이트를, 파일 ip의 현재 오프셋(off)로부터 차례대로 쓰되,
// 각 파일 블록에 대해 COW가 필요하면 새 블록을 하나 할당해서 기존 내용을 복사한 뒤 그 새 블록에 써 넣는 로직이다
// 각 파일 블록을 쓰기 전에 need_cow()로 스냅샷 공유 여부를 검사.
// 공유 중이라면 새 블록을 할당해서 old 내용을 복사한 뒤 inode 매핑을 새 블록으로 교체
int
writei(struct inode *ip, char *src, uint off, uint n)
{
    uint tot, m;

    if(ip->type == T_DEV){
        if(ip->major < 0 || ip->major >= NDEV || !devsw[ip->major].write)
            return -1;
        return devsw[ip->major].write(ip, src, n);
    }

    if(off > ip->size || off + n < off)
        return -1;
    if(off + n > MAXFILE*BSIZE)
        return -1;

    // n: 전체로 쓰고 싶은 바이트 수, tot: 지금까지 쓴 전체 바이트 수(누적)
    // off: 파일 내 현재 쓰기 위치(파일 오프셋), src: 사용자 버퍼의 현재 위치
    // 블록 단위로 나누어 쓰는 루프
    for(tot=0; tot<n; tot+=m, off+=m, src+=m){

        // 1. 이 오프셋(off)이 속한 파일 블록 찾기
        uint bn = off / BSIZE; // 현재 오프셋 off가 파일의 몇 번째 논리 블록에 속하는지 계산(bn = 데이터를 쓸 위치가 속한
        // 파일 블록 번호)
        uint bno = bmap(ip, bn); // (해당 논리 블록이 없으면 새로 할당됨: refcnt=1, bno=실제 디스크 블록 번호)

        // COW 필요 여부 체크
        // need_cow()를 이용해서 디스크 블록 bno가 스냅샷과 공유 중인지 확인한다.
        // need_cow()가 true라면 이 블록을 스냅샷도 쓰고 있으므로 COW를 진행한다.
        if(need_cow(bno)){
            uint newb = balloc(ip->dev); // 새 블록 할당(refcnt(newb)=1)

```

```

    struct buf *obp = bread(ip->dev, bno); // old block 읽기
    struct buf *nbp = bread(ip->dev, newb); // new block 버퍼
    memmove(nbp->data, obp->data, BSIZE); // old->new 전체 복사(기존 블록 bno의 내용을 통째로 newb 블록으로 복사)
    log_write(nbp); // 로그 시스템에 변경된 newb 버퍼를
기록(저널링)
    brelse(obp); // 버퍼 사용 끝났으니 해제
    brelse(nbp);

//      cprintf("[DEBUG] [COW] inum = %d bn = %d old = %x -> new = %x\n", ip->inum, bn, bno, newb);

// inode 매핑 교체
// inode ip에서 논리 블록 bn이 가리키는 디스크 블록을 newb로 바꾸는 작업을 진행한다
set_block(ip, bn, newb);

// old는 참조 감소 (스냅샷 pin이 0일 때만 실제 해제됨)
// 즉, 기존 블록은 참조 감소한다.
bdecref(ip->dev, bno);

// 이후 로직에서는 새 블록(newb) 사용
bno = newb;
}

// 실제 데이터 쓰기
struct buf *bp = bread(ip->dev, bno); // 지금 쓰려는 디스크 블록 bno를 버퍼로 읽어온다
m = min(n - tot, BSIZE - off%BSIZE); // 이번에 블록에 쓸 바이트 수 계산. min(아직 안 쓴 남은 전체 바이트
수, 현재 블록에서 남아 있는 공간)
memmove(bp->data + off%BSIZE, src, m); // 메모리->버퍼로 데이터 복사
log_write(bp); // 이 버퍼의 변경 내용을 로그에 기록(저널링)
brelse(bp); // 버퍼 사용 끝
}

// 파일 크기 갱신
// 이때 이번 write 때문에 파일이 더 커졌으면 inode의 파일 크기도 같이 키운다.
if(n > 0 && off > ip->size){
    ip->size = off;
    iupdate(ip);
}
return n;
}

//PAGEBREAK!
// Directories

int
namecmp(const char *s, const char *t)
{
    return strncmp(s, t, DIRSIZ);
}

// Look for a directory entry in a directory.
// If found, set *poff to byte offset of entry.
struct inode*
dirlookup(struct inode *dp, char *name, uint *poff)
{
    uint off, inum;
    struct dirent de;

    if(dp->type != T_DIR)

```

```

    panic("dirlookup not DIR");

for(off = 0; off < dp->size; off += sizeof(de)){
    if(readi(dp, (char*)&de, off, sizeof(de)) != sizeof(de))
        panic("dirlookup read");
    if(de.inum == 0)
        continue;
    if(namecmp(name, de.name) == 0){
        // entry matches path element
        if(poff)
            *poff = off;
        inum = de.inum;
        return iget(dp->dev, inum);
    }
}

return 0;
}

// Write a new directory entry (name, inum) into the directory dp.
int
dirlink(struct inode *dp, char *name, uint inum)
{
    int off;
    struct dirent de;
    struct inode *ip;

    // Check that name is not present.
    if((ip = dirlookup(dp, name, 0)) != 0){
        iput(ip);
        return -1;
    }

    // Look for an empty dirent.
    for(off = 0; off < dp->size; off += sizeof(de)){
        if(readi(dp, (char*)&de, off, sizeof(de)) != sizeof(de))
            panic("dirlink read");
        if(de.inum == 0)
            break;
    }

    strncpy(de.name, name, DIRSIZ);
    de.inum = inum;
    if(writei(dp, (char*)&de, off, sizeof(de)) != sizeof(de))
        panic("dirlink");

    return 0;
}

//PAGEBREAK!
// Paths

// Copy the next path element from path into name.
// Return a pointer to the element following the copied one.
// The returned path has no leading slashes,
// so the caller can check *path=='\0' to see if the name is the last one.
// If no name to remove, return 0.
//

```

```

// Examples:
//  skipelem("a/bb/c", name) = "bb/c", setting name = "a"
//  skipelem("///a//bb", name) = "bb", setting name = "a"
//  skipelem("a", name) = "", setting name = "a"
//  skipelem("", name) = skipelem("///", name) = 0
//
static char*
skipelem(char *path, char *name)
{
    char *s;
    int len;

    while(*path == '/')
        path++;
    if(*path == 0)
        return 0;
    s = path;
    while(*path != '/' && *path != 0)
        path++;
    len = path - s;
    if(len >= DIRSIZ)
        memmove(name, s, DIRSIZ);
    else {
        memmove(name, s, len);
        name[len] = 0;
    }
    while(*path == '/')
        path++;
    return path;
}

// Look up and return the inode for a path name.
// If parent != 0, return the inode for the parent and copy the final
// path element into name, which must have room for DIRSIZ bytes.
// Must be called inside a transaction since it calls iput().
static struct inode*
namex(char *path, int nameiparent, char *name)
{
    struct inode *ip, *next;

    if(*path == '/')
        ip = iget(ROOTDEV, ROOTINO);
    else
        ip = idup(myproc()->cwd);

    while((path = skipelem(path, name)) != 0){
        ilock(ip);
        if(ip->type != T_DIR){
            iunlockput(ip);
            return 0;
        }
        if(nameiparent && *path == '\0'){
            // Stop one level early.
            iunlock(ip);
            return ip;
        }
        if((next = dirlookup(ip, name, 0)) == 0){
            iunlockput(ip);

```



```

    return 0;
}
iunlockput(ip);
ip = next;
}
if(nameiparent){
    iput(ip);
    return 0;
}
return ip;
}

struct inode*
namei(char *path)
{
    char name[DIRSIZ];
    return namex(path, 0, name);
}

struct inode*
nameiparent(char *path, char *name)
{
    return namex(path, 1, name);
}

// 디스크 블록 b에 대해 COW가 필요한 상황인지 체크하는 함수
// 블록 b를 덮어쓰기 전에, 이 블록이 스냅샷과 공유 중이면 COW를 해야 하고, 아니면 그냥 덮어쓴다
// snap>0 -> 1: COW가 필요하다
// snap<=0 -> 0: COW가 필요 없다(그냥 덮어써도 됨)
static int
need_cow(uint b)
{
    int snap;
    acquire(&block_meta_lock);

    // b번 블록을 현재 몇 개의 스냅샷이 참조 중인지 읽어온다
    // 예: snap == 2 -> 이 블록을 참조하는 스냅샷이 2개 있음
    snap = block_snapcnt[b];
    release(&block_meta_lock);
    return snap > 0;
}

// 파일의 bn번째 데이터 블록이 실제 디스크의 어느 블록(newb)에 저장되는지 설정해주는 함수
// 즉, inode 안에 있는 블록 번호 테이블(직접/간접 블록)을 수정하는 역할을 한다.
// ip: 어느 파일(inode)에 대해 작업할지, bn: 그 파일 안에서의 블록 번호(0,1,2,... 같은 논리적 블록 인덱스)
// newb: 이 논리 블록이 실제 디스크에서 가리킬 디스크 블록 번호
// 디스크에서 inum에 해당하는 dinode를 직접 읽어온다 (락 없음)
static void
set_block(struct inode *ip, uint bn, uint newb)
{
    // 1) 직접 블록 영역
    if(bn < NDIRECT){
        ip->addrs[bn] = newb;
        iupdate(ip);
        return;
    }

    // 2) 간접 블록 영역으로 인덱스 조정

```

```

bn -= NDIRECT;

if(bn < NINDIRECT){
    uint ib = ip->addrs[NDIRECT];

    if(ib == 0)
        panic("set_block: no indirect");

    // * 간접 블록 자체에 대해 COW 필요 여부 검사
    if(need_cow(ib)){

        // (1) 새 간접 블록 할당
        uint newib = balloc(ip->dev);

        // 디버그용 출력(제출 전에 지우기)
        // cprintf("[DEBUG] [COW-INDIRECT] inum=%d old_ib=%x -> new_ib=%x\n", ip->inum, ib, newib);

        // (2) 기존 간접 블록 내용을 새 블록으로 복사
        struct buf *obp = bread(ip->dev, ib);
        struct buf *nbp = bread(ip->dev, newib);
        memmove(nbp->data, obp->data, BSIZE);
        log_write(nbp);
        brelse(obp);
        brelse(nbp);

        // (3) inode가 가리키는 간접 블록을 새 블록으로 교체
        ip->addrs[NDIRECT] = newib;
        iupdate(ip);          // 디스크의 inode도 갱신

        // (4) 이전 간접 블록에 대한 참조 감소
        bdecref(ip->dev, ib);

        // 이후 로직에서는 새 간접 블록을 사용
        ib = newib;
    }

    // 여기서부터는 "이 inode만 사용하는 간접 블록 ib"를 수정하는 부분
    struct buf *bp = bread(ip->dev, ib);
    uint *a = (uint*)bp->data;

    a[bn] = newib;          // bn번째 엔트리를 newb로 교체

    log_write(bp);
    brelse(bp);
    return;
}

panic("set_block: out of range");
}

// 락 없이 디스크의 dinode 한 개 읽기
// rebuild_block_meta에서 inode 전체를 순회할 때 사용
static void
read_dinode_nolock(int dev, uint inum, struct dinode *out)
{
    struct buf *bp = bread(dev, IBLOCK(inum, sb));

```

```

struct dinode *dip = (struct dinode*)bp->data + inum % IPB;
memmove(out, dip, sizeof(*out));
brelse(bp);
}

// Free a disk block.
// orphan 정리 전용. 기존 xv6의 bfree를 이름만 달리 복사
// cleanup_orphan_blocks()에서 refcnt/snapcnt==0인 블록을 직접 free할때만 사용한다
static void
bfree(int dev, uint b)
{
    struct buf *bp;
    int bi, m;

    bp = bread(dev, BBLOCK(b, sb));
    bi = b % BPB;
    m = 1 << (bi % 8);
    if((bp->data[bi/8] & m) == 0)
        panic("freeing free block");
    bp->data[bi/8] &= ~m;
    log_write(bp);
    brelse(bp);
}

// 디스크 비트맵과 block_refcnt / block_snapcnt를 비교해서
// "아무도 참조하지 않는데(refcnt == 0 && snapcnt == 0)
// 비트맵에는 할당된 것으로 표시된 데이터 블록"을 찾아 실제로 bfree()로 해제하는 함수.
// => 내부에서 begin_op ~ end_op 를 직접 관리한다.
// (호출하는 쪽에서는 트랜잭션을 열지 말 것!)
void
cleanup_orphan_blocks(int dev)
{
    int b;
    int freed_in_tx = 0;

    // 메타데이터 블록 개수 = 전체 - 데이터블록
    // 데이터 블록은 [first_data, sb.size) 구간에만 존재
    int first_data = sb.size - sb.nblocks;

    begin_op(); // 첫 트랜잭션 시작

    // 메타데이터(0 .. first_data-1)는 건너뛰고
    // 오직 "데이터 블록" 구간만 검사한다.
    for (b = first_data; b < sb.size; b++) {
        int bi = b % BPB;
        int m = 1 << (bi % 8);

        // 비트맵에서 이 블록이 할당 상태인지 확인
        struct buf *bp = bread(dev, BBLOCK(b, sb));
        int allocated = bp->data[bi/8] & m;
        brelse(bp);

        if (!allocated)
            continue;

        int ref, snap;
        acquire(&block_meta_lock);
        ref = block_refcnt[b];

```

```

snap = block_snapcnt[b];
release(&block_meta_lock);

// 어떤 파일/스냅샷에서도 참조하지 않는 "데이터 블록"만 free
if (ref == 0 && snap == 0) {
    bfree(dev, b);    // 항상 begin_op~end_op 안

    freed_in_tx++;
    // 한 트랜잭션에서 너무 많은 free를 하지 않도록 쪼개기
    if (freed_in_tx >= MAX_FREE_PER_TX) {
        end_op();      // 지금까지 free 한 것 commit
        begin_op();    // 새 트랜잭션 시작
        freed_in_tx = 0;
    }
}

end_op();    // 마지막 트랜잭션 종료
}

static void
blockmeta_init(int dev)
{
    struct inode *snapdir;
    struct inode *ip;
    // char name[DIRSIZ];

    // /snapshot/blockmeta 를 찾아온다.
    // 이미 mkfs에서 만들어졌다고 가정.
    begin_op();
    snapdir = namei("/snapshot");
    if(snapdir == 0){
        end_op();
        panic("blockmeta_init: no /snapshot");
    }

    // /snapshot 디렉토리 안에서 blockmeta 찾기
    ilock(snapdir);
    ip = dirlookup(snapdir, "blockmeta", 0);
    iunlockput(snapdir);
    end_op();

    if(ip == 0)
        panic("blockmeta_init: no /snapshot/blockmeta");

    // 전역 포인터에 저장
    blockmeta_ip = ip;

    // 첫 부팅 또는 crash 후 재부팅 시점에
    // 메모리에서 다시 계산된 ref/snap을 blockmeta 파일로 전체 덮어쓰기
    // blockmeta_full_sync();
}

/*
static void
blockmeta_full_sync(void)
{
    if(blockmeta_ip == 0)

```

```

panic("blockmeta_full_sync: no blockmeta_ip");

struct block_meta m;
uint off = 0;
uint b;
int cnt_in_tx = 0;

begin_op();
ilock(blockmeta_ip);

for (b = 0; b < sb.size && b < FSSIZE; b++) {

    acquire(&block_meta_lock);
    m.ref = (uchar)block_refcnt[b];
    m.snap = (uchar)block_snapcnt[b];
    release(&block_meta_lock);

    if (writei(blockmeta_ip, (char*)&m,
               off, sizeof(m)) != sizeof(m))
        panic("blockmeta_full_sync: writei");
    off += sizeof(m);
    cnt_in_tx++;

    // 로그에 너무 많은 블록이 쌓이지 않도록 중간중간 커밋
    if (cnt_in_tx >= BLOCKMETA_CHUNK) {
        iunlock(blockmeta_ip);
        end_op();          // 지금까지 기록한 것 커밋

        begin_op();        // 새 트랜잭션 시작
        ilock(blockmeta_ip);
        cnt_in_tx = 0;
    }
}

iunlock(blockmeta_ip);
end_op();
}
*/

static void
blockmeta_write_record(uint b, int ref, int snap)
{
    if(blockmeta_ip == 0)
        return;    // 아직 초기화 안 되었으면 조용히 무시 (부팅 초기 단계 대비)

    if(b >= FSSIZE)
        return;

    struct block_meta m;
    m.ref = (uchar)ref;
    m.snap = (uchar)snap;

    // 이미 sys_*에서 begin_op()가 열려 있고,
    // 그 안에서 balloc/bdecref/writei 등이 돌고 있다는 전제하에,
    // 여기서는 별도 begin_op() 없이 writei만 호출한다.
    ilock(blockmeta_ip);
    if(writei(blockmeta_ip, (char*)&m,
              b * sizeof(struct block_meta),

```

```

        sizeof(struct block_meta)) != sizeof(struct block_meta))
    panic("blockmeta_write_record: write!");
iunlock(blockmeta_ip);
}

// -----
// /snapshot/blockmeta in-place 갱신 헬퍼들
// - blockmeta_add_ref()      : refcnt++, (is_snap!=0이면 snapcnt++)
// - blockmeta_add_ref_nosnap : refcnt++
// - blockmeta_dec_snap()     : snapcnt--
// -----

// refcnt[b]++, 그리고 is_snap != 0 이면 snapcnt[b]++ 까지 수행한 뒤,
// /snapshot/blockmeta 의 b 번째 레코드를 in-place 갱신한다.
//
void
blockmeta_add_ref(uint b, int is_snap)
{
    int ref, snap;

    acquire(&block_meta_lock);
    if(b >= FSSIZE)
        panic("blockmeta_add_ref: b out of range");

    block_refcnt[b]++;
    ref = block_refcnt[b];

    if(is_snap){
        block_snapcnt[b]++;
    }
    snap = block_snapcnt[b];
    release(&block_meta_lock);

    // 기존 3인자 버전 helper 호출
    blockmeta_write_record(b, ref, snap);
}

// 일반 파일용: refcnt[b]++ 만 수행하고 디스크 갱신
void
blockmeta_add_ref_nosnap(uint b)
{
    blockmeta_add_ref(b, 0);
}

// 스냅샷이 없어질 때(스냅샷 파일 삭제 시),
// T_SNAP inode가 참조하던 블록에 대해 snapcnt[b]-- 만 수행.
// refcnt 감소는 itrunc -> bdecref 경로에서 처리.
//
void
blockmeta_dec_snap(uint b)
{
    int ref, snap;

    acquire(&block_meta_lock);
    if(b >= FSSIZE)
        panic("blockmeta_dec_snap: b out of range");
    if(block_snapcnt[b] <= 0)
        panic("blockmeta_dec_snap: underflow");

```

```

    block_snapcnt[b]--;
    ref = block_refcnt[b];
    snap = block_snapcnt[b];
    release(&block_meta_lock);

    blockmeta_write_record(b, ref, snap);
}
11. fs.h
// On-disk file system format.
// Both the kernel and user programs use this header file.

#define ROOTINO 1 // root i-number
#define BSIZE 512 // block size

// Disk layout:
// [ boot block | super block | log | inode blocks |
//                                free bit map | data blocks]
//
// mkfs computes the super block and builds an initial file system. The
// super block describes the disk layout:
struct superblock {
    uint size; // Size of file system image (blocks)
    uint nblocks; // Number of data blocks
    uint ninodes; // Number of inodes.
    uint nlog; // Number of log blocks
    uint logstart; // Block number of first log block
    uint inodestart; // Block number of first inode block
    uint bmapstart; // Block number of first free map block
};

#define NDIRECT 12
#define NINDIRECT (BSIZE / sizeof(uint))
#define MAXFILE (NDIRECT + NINDIRECT)

// On-disk inode structure
struct dinode {
    short type; // File type
    short major; // Major device number (T_DEV only)
    short minor; // Minor device number (T_DEV only)
    short nlink; // Number of links to inode in file system
    uint size; // Size of file (bytes)
    uint addrs[NDIRECT+1]; // Data block addresses
};

// Inodes per block.
#define IPB (BSIZE / sizeof(struct dinode))

// Block containing inode i
#define IBLOCK(i, sb) ((i) / IPB + sb.inodestart)

// Bitmap bits per block
#define BPB (BSIZE*8)

// Block of free map containing bit for block b
#define BBLOCK(b, sb) (b/BPB + sb.bmapstart)

```

```
// Directory is a file containing a sequence of dirent structures.
#define DIRSIZ 14

struct dirent {
    ushort inum;
    char name[DIRSIZ];
};

// 설계4를 위한 추가 함수
void rebuild_block_meta(int dev);
struct inode* iget(uint dev, uint inum);

struct block_meta {
    uchar ref;
    uchar snap;
};

#define BLOCKMETA_PATH "/snapshot/blockmeta"
#define BLOCKMETA_RECORD_SIZE (sizeof(struct block_meta))
#define BLOCKMETA_RPB (BSIZE / sizeof(struct block_meta)) // Records Per Block
```

12. mkfs.c

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <assert.h>

#define stat xv6_stat // avoid clash with host struct stat
#include "types.h"
#include "fs.h"
#include "stat.h"
#include "param.h"

#ifndef static_assert
#define static_assert(a, b) do { switch (0) case 0: case (a): ; } while (0)
#endif

#define NINODES 200

// Disk layout:
// [ boot block | sb block | log | inode blocks | free bit map | data blocks ]

int nbitmap = FSSIZE/(BSIZE*8) + 1;
int ninodeblocks = NINODES / IPB + 1;
int nlog = LOGSIZE;
int nmeta; // Number of meta blocks (boot, sb, nlog, inode, bitmap)
int nblocks; // Number of data blocks

int fsfd;
struct superblock sb;
char zeroes[BSIZE];
uint freeinode = 1;
uint freeblock;
```



```

void balloc(int);
void wsect(uint, void*);
void winode(uint, struct dinode*);
void rinode(uint inum, struct dinode *ip);
void rsect(uint sec, void *buf);
uint ialloc(ushort type);
void iappend(uint inum, void *p, int n);


// convert to intel byte order
ushort
xshort(ushort x)
{
    ushort y;
    uchar *a = (uchar*)&y;
    a[0] = x;
    a[1] = x >> 8;
    return y;
}

uint
xint(uint x)
{
    uint y;
    uchar *a = (uchar*)&y;
    a[0] = x;
    a[1] = x >> 8;
    a[2] = x >> 16;
    a[3] = x >> 24;
    return y;
}

int
main(int argc, char *argv[])
{
    int i, cc, fd;
    uint rootino, inum, off;
    struct dirent de;
    char buf[BSIZE];
    struct dinode din;

    static_assert(sizeof(int) == 4, "Integers must be 4 bytes!");

    if(argc < 2){
        fprintf(stderr, "Usage: mkfs fs.img files...\n");
        exit(1);
    }

    assert((BSIZE % sizeof(struct dinode)) == 0);
    assert((BSIZE % sizeof(struct dirent)) == 0);

    fsfd = open(argv[1], O_RDWR|O_CREAT|O_TRUNC, 0666);
    if(fsfd < 0){
        perror(argv[1]);
        exit(1);
    }

    // 1 fs block = 1 disk sector

```

```

nmeta = 2 + nlog + ninodeblocks + nbitmap;
nblocks = FSSIZE - nmeta;

sb.size      = xint(FSSIZE);
sb.nblocks   = xint(nblocks);
sb.ninodes   = xint(NINODES);
sb.nlog      = xint(nlog);
sb.logstart  = xint(2);
sb.inodestart= xint(2+nlog);
sb.bmapstart = xint(2+nlog+ninodeblocks);

printf("nmeta %d (boot, super, log blocks %u inode blocks %u, bitmap blocks %u) blocks %d total %d\n",
       nmeta, nlog, ninodeblocks, nbitmap, nblocks, FSSIZE);

// 메타 블록(boot, super, log, inode, bitmap)까지는 이미 예약된 상태
freeblock = nmeta;    // data block 중 첫 free 블록 번호

// 디스크 전체 0으로 초기화
for(i = 0; i < FSSIZE; i++)
    wsect(i, zeroes);

// superblock 기록
memset(buf, 0, sizeof(buf));
memmove(buf, &sb, sizeof(sb));
wsect(1, buf);

// 루트 디렉토리 inode 할당
rootino = ialloc(T_DIR);
assert(rootino == ROOTINO);

// 루트 디렉토리 엔트리 "." 추가
bzero(&de, sizeof(de));
de.inum = xshort(rootino);
strcpy(de.name, ".");
iappend(rootino, &de, sizeof(de));

// 루트 디렉토리 엔트리 ".." 추가
bzero(&de, sizeof(de));
de.inum = xshort(rootino);
strcpy(de.name, "..");
iappend(rootino, &de, sizeof(de));

// -----
// argv[2..] 유저 프로그램 파일들 복사 (기존 xv6 mkfs 코드 그대로)
// -----
for(i = 2; i < argc; i++){
    assert(index(argv[i], '/') == 0);

    if((fd = open(argv[i], 0)) < 0){
        perror(argv[i]);
        exit(1);
    }

    // 파일 이름이 _rm, _cat 처럼 _로 시작하면, 파일시스템에는 rm, cat으로 저장
    if(argv[i][0] == '_')
        ++argv[i];

    inum = ialloc(T_FILE);

```

```

bzero(&de, sizeof(de));
de.inum = xshort(inum);
strncpy(de.name, argv[i], DIRSIZ);
iappend(rootino, &de, sizeof(de));

while((cc = read(fd, buf, sizeof(buf))) > 0)
    iappend(inum, buf, cc);

close(fd);
}

// -----
// 여기부터 새로 추가: /snapshot 디렉토리 & /snapshot/blockmeta 파일 생성
// -----

// 1) /snapshot 디렉토리 inode 할당
uint snapdir_ino = ialloc(T_DIR);

// 루트 디렉토리에 "snapshot" 엔트리 추가
bzero(&de, sizeof(de));
de.inum = xshort(snapdir_ino);
strcpy(de.name, "snapshot");
iappend(rootino, &de, sizeof(de));

// /snapshot/. 엔트리
bzero(&de, sizeof(de));
de.inum = xshort(snapdir_ino);
strcpy(de.name, ".");
iappend(snapdir_ino, &de, sizeof(de));

// /snapshot/.. 엔트리 (부모는 루트)
bzero(&de, sizeof(de));
de.inum = xshort(rootino);
strcpy(de.name, "..");
iappend(snapdir_ino, &de, sizeof(de));

// 2) /snapshot/blockmeta 파일 inode 할당
uint meta_ino = ialloc(T_FILE);

// /snapshot 디렉토리에 "blockmeta" 엔트리 추가
bzero(&de, sizeof(de));
de.inum = xshort(meta_ino);
strcpy(de.name, "blockmeta");
iappend(snapdir_ino, &de, sizeof(de));

// 3) /snapshot/blockmeta 파일 내용 채우기
//
//   - block 0 ~ FSSIZE-1 까지 각 블록에 대해 struct block_meta 한 개씩
//   - 즉, 총 FSSIZE 개 레코드
//   - 레코드 배열을 BSIZE 단위로 잘라서 iappend로 쓰기
//
int n_records = FSSIZE;
int n_meta_blocks = (n_records + BLOCKMETA_RPB - 1) / BLOCKMETA_RPB;

printf("mkfs: creating /snapshot/blockmeta with %d records, %d blocks\n",
       n_records, n_meta_blocks);

```

```

for(int bi = 0; bi < n_meta_blocks; bi++){
    struct block_meta mbuf[BLOCKMETA_RPB];
    memset(mbuf, 0, sizeof(mbuf));

    // 항상 BSIZE 만큼 append (mbuf 전체)
    iappend(meta_ino, (char*)mbuf, BSIZE);
}

// -----
// 새로 추가 끝
// -----

// 루트 디렉토리 크기를 BSIZE 배수로 맞춤
// 위에서 "snapshot" 엔트리까지 이미 다 추가된 상태의 size 기준
rinode(rootino, &din);
off = xint(din.size);
off = ((off/BSIZE) + 1) * BSIZE;
din.size = xint(off);
winode(rootino, &din);

// 지금까지 사용한 데이터 블록 수(freeblock)를 기준으로
// 비트맵 블록(bmapstart)에 할당 비트 설정
balloc(freeblock);

exit(0);
}

```

```

void
wsect(uint sec, void *buf)
{
    if(lseek(fsfd, sec * BSIZE, 0) != sec * BSIZE){
        perror("lseek");
        exit(1);
    }
    if(write(fsfd, buf, BSIZE) != BSIZE){
        perror("write");
        exit(1);
    }
}

```

```

void
winode(uint inum, struct dinode *ip)
{
    char buf[BSIZE];
    uint bn;
    struct dinode *dip;

    bn = IBLOCK(inum, sb);
    rsect(bn, buf);
    dip = ((struct dinode*)buf) + (inum % IPB);
    *dip = *ip;
    wsect(bn, buf);
}

```

```

void
rinode(uint inum, struct dinode *ip)
{

```

```

char buf[BSIZE];
uint bn;
struct dinode *dip;

bn = IBLOCK(inum, sb);
rsect(bn, buf);
dip = ((struct dinode*)buf) + (inum % IPB);
*ip = *dip;
}

void
rsect(uint sec, void *buf)
{
    if(lseek(fsfd, sec * BSIZE, 0) != sec * BSIZE){
        perror("lseek");
        exit(1);
    }
    if(read(fsfd, buf, BSIZE) != BSIZE){
        perror("read");
        exit(1);
    }
}

uint
ialloc(ushort type)
{
    uint inum = freeinode++;
    struct dinode din;

    bzero(&din, sizeof(din));
    din.type = xshort(type);
    din.nlink = xshort(1);
    din.size = xint(0);
    winode(inum, &din);
    return inum;
}

void
balloc(int used)
{
    uchar buf[BSIZE];
    int i;

    printf("balloc: first %d blocks have been allocated\n", used);
    assert(used < BSIZE*8);
    bzero(buf, BSIZE);
    for(i = 0; i < used; i++){
        buf[i/8] = buf[i/8] | (0x1 << (i%8));
    }
    printf("balloc: write bitmap block at sector %d\n", sb.bmapstart);
    wsect(sb.bmapstart, buf);
}

#define min(a, b) ((a) < (b) ? (a) : (b))

void
iappend(uint inum, void *xp, int n)
{

```

```

char *p = (char*)xp;
uint fbn, off, n1;
struct dinode din;
char buf[BSIZE];
uint indirect[NINDIRECT];
uint x;

rinode(inum, &din);
off = xint(din.size);
// printf("append inum %d at off %d sz %d\n", inum, off, n);
while(n > 0){
    fbn = off / BSIZE;
    assert(fbn < MAXFILE);
    if(fbn < NDIRECT){
        if(xint(din.addrs[fbn]) == 0){
            din.addrs[fbn] = xint(freeblock++);
        }
        x = xint(din.addrs[fbn]);
    } else {
        if(xint(din.addrs[NDIRECT]) == 0){
            din.addrs[NDIRECT] = xint(freeblock++);
        }
        rsect(xint(din.addrs[NDIRECT]), (char*)indirect);
        if(indirect[fbn - NDIRECT] == 0){
            indirect[fbn - NDIRECT] = xint(freeblock++);
            wsect(xint(din.addrs[NDIRECT]), (char*)indirect);
        }
        x = xint(indirect[fbn-NDIRECT]);
    }
    n1 = min(n, (fbn + 1) * BSIZE - off);
    rsect(x, buf);
    bcopy(p, buf + off - (fbn * BSIZE), n1);
    wsect(x, buf);
    n -= n1;
    off += n1;
    p += n1;
}
din.size = xint(off);
winode(inum, &din);
}

```

13. stat.h

```

#define T_DIR 1 // Directory
#define T_FILE 2 // File
#define T_DEV 3 // Device
#define T_SNAP 4 // Snapshot을 위한 타입 추가 구현

```

```

struct stat {
    short type; // Type of file
    int dev; // File system's disk device
    uint ino; // Inode number
    short nlink; // Number of links to file
    uint size; // Size of file in bytes
};

```

14. string.c

```

#include "types.h"
#include "x86.h"

```

```

void*

```

```

memset(void *dst, int c, uint n)
{
    if ((int)dst%4 == 0 && n%4 == 0){
        c &= 0xFF;
        stosl(dst, (c<<24)|(c<<16)|(c<<8)|c, n/4);
    } else
        stosb(dst, c, n);
    return dst;
}

int
memcmp(const void *v1, const void *v2, uint n)
{
    const uchar *s1, *s2;

    s1 = v1;
    s2 = v2;
    while(n-- > 0){
        if(*s1 != *s2)
            return *s1 - *s2;
        s1++, s2++;
    }

    return 0;
}

void*
memmove(void *dst, const void *src, uint n)
{
    const char *s;
    char *d;

    s = src;
    d = dst;
    if(s < d && s + n > d){
        s += n;
        d += n;
        while(n-- > 0)
            *--d = *--s;
    } else
        while(n-- > 0)
            *d++ = *s++;

    return dst;
}

// memcpy exists to placate GCC. Use memmove.
void*
memcpy(void *dst, const void *src, uint n)
{
    return memmove(dst, src, n);
}

int
strncmp(const char *p, const char *q, uint n)
{
    while(n > 0 && *p && *p == *q)
        n--, p++, q++;

```

```

    if(n == 0)
        return 0;
    return (uchar)*p - (uchar)*q;
}

```

```

char*
strncpy(char *s, const char *t, int n)
{
    char *os;

    os = s;
    while(n-- > 0 && (*s++ = *t++) != 0)
        ;
    while(n-- > 0)
        *s++ = 0;
    return os;
}

```

// Like strncpy but guaranteed to NUL-terminate.

```

char*
safestrcpy(char *s, const char *t, int n)
{
    char *os;

    os = s;
    if(n <= 0)
        return os;
    while(--n > 0 && (*s++ = *t++) != 0)
        ;
    *s = 0;
    return os;
}

```

```

int
strlen(const char *s)
{
    int n;

    for(n = 0; s[n]; n++)
        ;
    return n;
}

```

```

// 추가 구현
// 두 문자를 사전순으로 비교해서 같으면 0,
// p > q: 양수
// p < q: 음수를 반환
// 즉 두 문자열의 앞에서부터 한 글자씩 비교하다가 처음 다른 지점 또는 문자열 끝에서 멈추고, 그 지점의 문자 차이를 uchar로
계산해서 문자열의 사전순 크고 작음을 알려준다
int
strcmp(const char *p, const char *q)
{
    while(*p && *p == *q){
        p++;
        q++;
    }
    return (uchar)*p - (uchar)*q;
}

```


15. syscall.c

```
#include "types.h"
#include "defs.h"
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "proc.h"
#include "x86.h"
#include "syscall.h"

// User code makes a system call with INT T_SYSCALL.
// System call number in %eax.
// Arguments on the stack, from the user call to the C
// library system call function. The saved user %esp points
// to a saved program counter, and then the first argument.

// Fetch the int at addr from the current process.
int
fetchint(uint addr, int *ip)
{
    struct proc *curproc = myproc();

    if(addr >= curproc->sz || addr+4 > curproc->sz)
        return -1;
    *ip = *(int*)(addr);
    return 0;
}

// Fetch the nul-terminated string at addr from the current process.
// Doesn't actually copy the string - just sets *pp to point at it.
// Returns length of string, not including nul.
int
fetchstr(uint addr, char **pp)
{
    char *s, *ep;
    struct proc *curproc = myproc();

    if(addr >= curproc->sz)
        return -1;
    *pp = (char*)addr;
    ep = (char*)curproc->sz;
    for(s = *pp; s < ep; s++){
        if(*s == 0)
            return s - *pp;
    }
    return -1;
}

// Fetch the nth 32-bit system call argument.
int
argint(int n, int *ip)
{
    return fetchint((myproc()->tf->esp) + 4 + 4*n, ip);
}

// Fetch the nth word-sized system call argument as a pointer
// to a block of memory of size bytes. Check that the pointer
// lies within the process address space.
```

```

int
argptr(int n, char **pp, int size)
{
    int i;
    struct proc *curproc = myproc();

    if(argint(n, &i) < 0)
        return -1;
    if(size < 0 || (uint)i >= curproc->sz || (uint)i+size > curproc->sz)
        return -1;
    *pp = (char*)i;
    return 0;
}

// Fetch the nth word-sized system call argument as a string pointer.
// Check that the pointer is valid and the string is nul-terminated.
// (There is no shared writable memory, so the string can't change
// between this check and being used by the kernel.)
int
argstr(int n, char **pp)
{
    int addr;
    if(argint(n, &addr) < 0)
        return -1;
    return fetchstr(addr, pp);
}

extern int sys_chdir(void);
extern int sys_close(void);
extern int sys_dup(void);
extern int sys_exec(void);
extern int sys_exit(void);
extern int sys_fork(void);
extern int sys_fstat(void);
extern int sys_getpid(void);
extern int sys_kill(void);
extern int sys_link(void);
extern int sys_mkdir(void);
extern int sys_mknod(void);
extern int sys_open(void);
extern int sys_pipe(void);
extern int sys_read(void);
extern int sys_sbrk(void);
extern int sys_sleep(void);
extern int sys_unlink(void);
extern int sys_wait(void);
extern int sys_write(void);
extern int sys_uptime(void);
extern int sys_snapshot_create(void);      // 추가
extern int sys_snapshot_rollback(void);    // 추가
extern int sys_snapshot_delete(void);      // 추가
extern int sys_print_addr(void); // print_addr용

static int (*syscalls[])(void) = {
    [SYS_fork]    sys_fork,
    [SYS_exit]    sys_exit,
    [SYS_wait]    sys_wait,
    [SYS_pipe]    sys_pipe,

```

```

[SYS_read]    sys_read,
[SYS_kill]    sys_kill,
[SYS_exec]    sys_exec,
[SYS_fstat]   sys_fstat,
[SYS_chdir]   sys_chdir,
[SYS_dup]     sys_dup,
[SYS_getpid]  sys_getpid,
[SYS_sbrk]    sys_sbrk,
[SYS_sleep]   sys_sleep,
[SYS_uptime]  sys_uptime,
[SYS_open]    sys_open,
[SYS_write]   sys_write,
[SYS_mknod]   sys_mknod,
[SYS_unlink]  sys_unlink,
[SYS_link]    sys_link,
[SYS_mkdir]   sys_mkdir,
[SYS_close]   sys_close,
[SYS_snapshot_create] sys_snapshot_create,
[SYS_snapshot_rollback] sys_snapshot_rollback,
[SYS_snapshot_delete] sys_snapshot_delete,
[SYS_print_addr] sys_print_addr,
};

```

```

void
syscall(void)
{
    int num;
    struct proc *curproc = myproc();

    num = curproc->tf->eax;
    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
        curproc->tf->eax = syscalls[num]();
    } else {
        cprintf("%d %s: unknown sys call %d\n",
            curproc->pid, curproc->name, num);
        curproc->tf->eax = -1;
    }
}

```

16. syscall.h

```

// System call numbers
#define SYS_fork    1
#define SYS_exit    2
#define SYS_wait    3
#define SYS_pipe    4
#define SYS_read    5
#define SYS_kill    6
#define SYS_exec    7
#define SYS_fstat    8
#define SYS_chdir    9
#define SYS_dup     10
#define SYS_getpid   11
#define SYS_sbrk     12
#define SYS_sleep    13
#define SYS_uptime   14
#define SYS_open     15
#define SYS_write    16
#define SYS_mknod    17
#define SYS_unlink   18

```

```

#define SYS_link 19
#define SYS_mkdir 20
#define SYS_close 21
#define SYS_snapshot_create 22
#define SYS_snapshot_rollback 23
#define SYS_snapshot_delete 24
#define SYS_print_addr 25
17. sysfile.c
//
// File-system system calls.
// Mostly argument checking, since we don't trust
// user code, and calls into file.c and fs.c.
//

#include "types.h"
#include "defs.h"
#include "param.h"
#include "stat.h"
#include "mmu.h"
#include "proc.h"
#include "fs.h"
#include "spinlock.h"
#include "sleeplock.h"
#include "file.h"
#include "fcntl.h"

// print_addr를 위해서 추가
#include "buf.h"

// /snapshot/blockmeta 덤프 시 한 트랜잭션서 처리할 블록 개수
#define BLOCKMETA_CHUNK 16

// 스냅샷 ID 관리
static struct spinlock snap_lock;
static int snap_initd = 0;
static int next_snap_id = 1;

// 스냅샷/복구용 헬퍼 함수 프로토타입
static struct inode *create_subdir(struct inode *parent, char *name);
static struct inode *create_snapshot_file(struct inode *parent, char *name);
static struct inode *create_file(struct inode *parent, char *name);
static void dir_unlink_at(struct inode *dp, uint off, struct inode *ip);
static int snapshot_clone_dir(struct inode *src_dir, struct inode *dst_dir, int is_root);
static int rollback_clone_dir(struct inode *src_snap_dir, struct inode *dst_dir);
static int snapshot_remove_tree(struct inode *dir);
static int clear_root_except_snapshot(void);

// fs.c 에서 제공
extern void rebuild_block_meta(int dev);
extern void cleanup_orphan_blocks(int dev);
extern void itrunc(struct inode *ip); // 새로 추가
extern struct superblock sb; // sb.size 사용용
extern int block_refcnt[FSSIZE];
extern int block_snapcnt[FSSIZE];
extern struct spinlock block_meta_lock;

static void inc_snap_for_inode_blocks(struct inode *ip);

```

```

// refcnt[b]++, snapcnt[b]++ (is_snap != 0 인 경우, 즉 스냅샷이 생성되는 경우)
extern void blockmeta_add_ref(uint b, int is_snap);

// refcnt[b]++ (일반 파일용, snapcnt는 건드리지 않음)
extern void blockmeta_add_ref_nosnap(uint b);

// snapcnt[b]-- (스냅샷 파일이 사라질 때)
extern void blockmeta_dec_snap(uint b);

// 커널용 문자열 함수 프로토타입(string.c에 구현되어 있음)
int strcmp(const char*, const char*);

// safestrcpy와 비슷한 스타일의 safe strcat 구현
static void
safestrcat(char *dst, const char *src, int dstsz)
{
    int n = 0;

    // dst에서 이미 채워진 길이 찾기
    while(n < dstsz && dst[n] != 0)
        n++;

    // src를 dst 끝에 붙이되, 항상 마지막에 '\0' 보장
    for(int i = 0; src[i] && n + 1 < dstsz; i++, n++)
        dst[n] = src[i];

    dst[n] = 0;
}

// Fetch the nth word-sized system call argument as a file descriptor
// and return both the descriptor and the corresponding struct file.
static int
argfd(int n, int *pfd, struct file **pf)
{
    int fd;
    struct file *f;

    if(argint(n, &fd) < 0)
        return -1;
    if(fd < 0 || fd >= NOFILE || (f=myproc()->ofile[fd]) == 0)
        return -1;
    if(pfd)
        *pfd = fd;
    if(pf)
        *pf = f;
    return 0;
}

// Allocate a file descriptor for the given file.
// Takes over file reference from caller on success.
static int
fdalloc(struct file *f)
{
    int fd;
    struct proc *curproc = myproc();

```

```

for(fd = 0; fd < NOFILE; fd++){
    if(curproc->ofile[fd] == 0){
        curproc->ofile[fd] = f;
        return fd;
    }
}
return -1;
}

```

```

int
sys_dup(void)
{
    struct file *f;
    int fd;

    if(argfd(0, 0, &f) < 0)
        return -1;
    if((fd=fdalloc(f)) < 0)
        return -1;
    filedup(f);
    return fd;
}

```

```

int
sys_read(void)
{
    struct file *f;
    int n;
    char *p;

    if(argfd(0, 0, &f) < 0 || argint(2, &n) < 0 || argptr(1, &p, n) < 0)
        return -1;
    return fileread(f, p, n);
}

```

```

int
sys_write(void)
{
    struct file *f;
    int n;
    char *p;

    if(argfd(0, 0, &f) < 0 || argint(2, &n) < 0 || argptr(1, &p, n) < 0)
        return -1;
    return filewrite(f, p, n);
}

```

```

int
sys_close(void)
{
    int fd;
    struct file *f;

    if(argfd(0, &fd, &f) < 0)
        return -1;
    myproc()->ofile[fd] = 0;
    fileclose(f);
}

```

```

    return 0;
}

int
sys_fstat(void)
{
    struct file *f;
    struct stat *st;

    if(argfd(0, 0, &f) < 0 || argptr(1, (void*)&st, sizeof(*st)) < 0)
        return -1;
    return filestat(f, st);
}

// Create the path new as a link to the same inode as old.
int
sys_link(void)
{
    char name[DIRSIZ], *new, *old;
    struct inode *dp, *ip;

    if(argstr(0, &old) < 0 || argstr(1, &new) < 0)
        return -1;

    begin_op();
    if((ip = namei(old)) == 0){
        end_op();
        return -1;
    }

    ilock(ip);
    if(ip->type == T_DIR){
        iunlockput(ip);
        end_op();
        return -1;
    }

    ip->nlink++;
    iupdate(ip);
    iunlock(ip);

    if((dp = nameiparent(new, name)) == 0)
        goto bad;
    ilock(dp);
    if(dp->dev != ip->dev || dirlink(dp, name, ip->inum) < 0){
        iunlockput(dp);
        goto bad;
    }
    iunlockput(dp);
    iput(ip);

    end_op();

    return 0;

bad:
    ilock(ip);
    ip->nlink--;

```

```

iupdate(ip);
iunlockput(ip);
end_op();
return -1;
}

// Is the directory dp empty except for "." and ".." ?
static int
isdirempty(struct inode *dp)
{
    int off;
    struct dirent de;

    for(off=2*sizeof(de); off<dp->size; off+=sizeof(de)){
        if(readi(dp, (char*)&de, off, sizeof(de)) != sizeof(de))
            panic("isdirempty: readi");
        if(de.inum != 0)
            return 0;
    }
    return 1;
}

//PAGEBREAK!
int
sys_unlink(void)
{
    struct inode *ip, *dp;
    struct dirent de;
    char name[DIRSIZ], *path;
    uint off;

    if(argstr(0, &path) < 0)
        return -1;

    begin_op();
    if((dp = nameiparent(path, name)) == 0){
        end_op();
        return -1;
    }

    ilock(dp);

    // Cannot unlink "." or "..".
    if(namecmp(name, ".") == 0 || namecmp(name, "..") == 0)
        goto bad;

    if((ip = dirlookup(dp, name, &off)) == 0)
        goto bad;
    ilock(ip);

    if(ip->nlink < 1)
        panic("unlink: nlink < 1");
    if(ip->type == T_DIR && !isdirempty(ip)){
        iunlockput(ip);
        goto bad;
    }

    memset(&de, 0, sizeof(de));

```



```

if(writei(dp, (char*)&de, off, sizeof(de)) != sizeof(de))
    panic("unlink: writei");
if(ip->type == T_DIR){
    dp->nlink--;
    iupdate(dp);
}
iunlockput(dp);

ip->nlink--;
iupdate(ip);
iunlockput(ip);

end_op();

return 0;

bad:
iunlockput(dp);
end_op();
return -1;
}

static struct inode*
create(char *path, short type, short major, short minor)
{
    struct inode *ip, *dp;
    char name[DIRSIZ];

    if((dp = nameiparent(path, name)) == 0)
        return 0;
    ilock(dp);

    if((ip = dirlookup(dp, name, 0)) != 0){
        iunlockput(dp);
        ilock(ip);
        if(type == T_FILE && ip->type == T_FILE)
            return ip;
        iunlockput(ip);
        return 0;
    }

    if((ip = ialloc(dp->dev, type)) == 0)
        panic("create: ialloc");

    ilock(ip);
    ip->major = major;
    ip->minor = minor;
    ip->nlink = 1;
    iupdate(ip);

    if(type == T_DIR){ // Create . and .. entries.
        dp->nlink++; // for "."
        iupdate(dp);
        // No ip->nlink++ for ".": avoid cyclic ref count.
        if(dirlink(ip, ".", ip->inum) < 0 || dirlink(ip, "..", dp->inum) < 0)
            panic("create dots");
    }
}

```

```

if(dirlink(dp, name, ip->inum) < 0)
    panic("create: dirlink");

iunlockput(dp);

return ip;
}

int
sys_open(void)
{
    char *path;
    int fd, omode;
    struct file *f;
    struct inode *ip;

    if(argstr(0, &path) < 0 || argint(1, &omode) < 0)
        return -1;

    begin_op();

    if(omode & O_CREATE){
        ip = create(path, T_FILE, 0, 0);
        if(ip == 0){
            end_op();
            return -1;
        }
    } else {
        if((ip = namei(path)) == 0){
            end_op();
            return -1;
        }
        ilock(ip);
        // 1) 디렉토리면 쓰기 금지
        if(ip->type == T_DIR && omode != O_RDONLY){
            iunlockput(ip);
            end_op();
            return -1;
        }

        // 2) 스냅샷(T_SNAP) 파일은 읽기 전용으로만 open 허용
        if(ip->type == T_SNAP){
            // 쓰기 관련 모드가 하나라도 있으면 에러
            if(omode & (O_WRONLY | O_RDWR)){
                iunlockput(ip);
                end_op();
                return -1;
            }
            // 여기서 omode를 강제로 O_RDONLY로 normalize
            omode = O_RDONLY;
        }
    }

    if((f = filealloc()) == 0 || (fd = fdalloc(f)) < 0){
        if(f)
            fileclose(f);
        iunlockput(ip);
        end_op();
    }

```

```

        return -1;
    }
    iunlock(ip);
    end_op();

    f->type = FD_INODE;
    f->ip = ip;
    f->off = 0;
    f->readable = !(omode & O_WRONLY);
    f->writable = (omode & O_WRONLY) || (omode & O_RDWR);
    return fd;
}

int
sys_mkdir(void)
{
    char *path;
    struct inode *ip;

    begin_op();
    if(argstr(0, &path) < 0 || (ip = create(path, T_DIR, 0, 0)) == 0){
        end_op();
        return -1;
    }
    iunlockput(ip);
    end_op();
    return 0;
}

int
sys_mknod(void)
{
    struct inode *ip;
    char *path;
    int major, minor;

    begin_op();
    if((argstr(0, &path)) < 0 ||
        argint(1, &major) < 0 ||
        argint(2, &minor) < 0 ||
        (ip = create(path, T_DEV, major, minor)) == 0){
        end_op();
        return -1;
    }
    iunlockput(ip);
    end_op();
    return 0;
}

int
sys_chdir(void)
{
    char *path;
    struct inode *ip;
    struct proc *curproc = myproc();

    begin_op();
    if(argstr(0, &path) < 0 || (ip = namei(path)) == 0){

```

```

    end_op();
    return -1;
}
ilock(ip);
if(ip->type != T_DIR){
    iunlockput(ip);
    end_op();
    return -1;
}
iunlock(ip);
iput(curproc->cwd);
end_op();
curproc->cwd = ip;
return 0;
}

```

```

int
sys_exec(void)
{
    char *path, *argv[MAXARG];
    int i;
    uint uargv, uarg;

    if(argstr(0, &path) < 0 || argint(1, (int*)&uargv) < 0){
        return -1;
    }
    memset(argv, 0, sizeof(argv));
    for(i=0;; i++){
        if(i >= NELEM(argv))
            return -1;
        if(fetchint(uargv+4*i, (int*)&uarg) < 0)
            return -1;
        if(uarg == 0){
            argv[i] = 0;
            break;
        }
        if(fetchstr(uarg, &argv[i]) < 0)
            return -1;
    }
    return exec(path, argv);
}

```

```

int
sys_pipe(void)
{
    int *fd;
    struct file *rf, *wf;
    int fd0, fd1;

    if(argptr(0, (void*)&fd, 2*sizeof(fd[0])) < 0)
        return -1;
    if(pipealloc(&rf, &wf) < 0)
        return -1;
    fd0 = -1;
    if((fd0 = fdalloc(rf)) < 0 || (fd1 = fdalloc(wf)) < 0){
        if(fd0 >= 0)
            myproc()->ofile[fd0] = 0;
        fileclose(rf);
    }
}

```

```

        fileclose(wf);
        return -1;
    }
    fd[0] = fd0;
    fd[1] = fd1;
    return 0;
}

// 커널 내부에서 쓸 mkdir helper
static int
kern_mkdir(char *path)
{
    struct inode *ip;
    begin_op();
    ip = create(path, T_DIR, 0, 0);
    if(ip == 0) {
        end_op();
        return -1;
    }
    iunlockput(ip);
    end_op();
    return 0;
}

static int
ensure_snapshot_root(void)
{
    struct inode *ip;

    begin_op();
    ip = namei("/snapshot");
    if(ip == 0) {
        end_op();
        // 없다면 생성 시도
        if (kern_mkdir("/snapshot") < 0)
            return -1;
        return 0;
    } else {
        iput(ip); // namei() -> iput()
        end_op();
        return 0;
    }
}

static void
init_snap_once(void)
{
    if(!snap_init){
        initlock(&snap_lock, "snap_lock");
        snap_init = 1;
    }
}

// 10진수로 id를 문자열로 (유틸)
static void itoa_dec(int x, char *buf)
{
    char tmp[16];
    int n = 0;

```

```

if(x == 0){ buf[0]='0'; buf[1]=0; return; }
while(x > 0){ tmp[n++] = '0' + (x % 10); x /= 10; }
for(int i=0;i<n;i++) buf[i] = tmp[n-1-i];
buf[n] = 0;
}

// x를 16진수 문자열로 변환 (0x prefix 없이, %x와 같은 형식)
/*
static void
itoa_hex(uint x, char *buf)
{
    char tmp[16];
    int n = 0;

    if (x == 0) {
        buf[0] = '0';
        buf[1] = 0;
        return;
    }

    while (x > 0) {
        int d = x & 0xF;    // 16진수 한 자리
        if (d < 10)
            tmp[n++] = '0' + d;
        else
            tmp[n++] = 'a' + (d - 10); // 소문자 hex: a~f
        x >>= 4;
    }

    // 역순으로 뒤집어서 buf에 저장
    for (int i = 0; i < n; i++)
        buf[i] = tmp[n - 1 - i];
    buf[n] = 0;
}
*/

// snapshot_create는 static이 아니어야 함
// (sys_snapshot_create()에서 호출해야 하니까)
int
snapshot_create(void)
{
    init_snap_once();

    // 1) /snapshot 디렉토리 보장
    if (ensure_snapshot_root() < 0)
        return -1;

    // 2) 새 ID 배정
    int id;
    acquire(&snap_lock);
    id = next_snap_id++;
    release(&snap_lock);

    // 3) /snapshot/<id> 디렉토리 생성
    char name[16], path[32];
    itoa_dec(id, name);
    safestrcpy(path, "/snapshot/", sizeof(path));
    safestrcat(path, name, sizeof(path));

```

```

if (kern_mkdir(path) < 0)
    return -1;

// 4) 원본 루트와 새 루트를 inode로 얻음
begin_op();

struct inode *root = iget(ROOTDEV, ROOTINO);
struct inode *snap_root = namei(path); // 방금 만든 /snapshot/<id>

if(root == 0 || snap_root == 0){
    end_op();
    return -1;
}

ilock(root);
ilock(snap_root);

// 5) "/" 아래를 재귀 복사하되, /snapshot과 T_DEV는 제외
snapshot_clone_dir(root, snap_root, 1 /* is_root */);

iunlockput(snap_root);
iunlockput(root);

end_op();

return id;
}

```

// 현재 파일시스템을 /snapshot/<id> 상태로 롤백하는 함수
int

```

snapshot_rollback(int id)
{
    if(id < 0){
        return -1;
    }
    char name[16], path[32];
    itoa_dec(id, name);
    safestrcpy(path, "/snapshot/", sizeof(path));
    safestrcat(path, name, sizeof(path));

```

```

begin_op();

```

```

// /snapshot/<id> 찾기
struct inode *snap_root = namei(path);
if(snap_root == 0){
    end_op();
    return -1; // invalid id
}

```

```

// 1) 현재 루트 정리 (snapshot 디렉토리 제외)
clear_root_except_snapshot();

```

```

// 2) 스냅샷 내용을 루트로 복사
struct inode *root = iget(ROOTDEV, ROOTINO);
ilock(root);

```

```

ilock(snap_root);

rollback_clone_dir(snap_root, root);

iunlockput(snap_root);
iunlockput(root);

end_op();

return 0;
}

// /snapshot/<id> 트리 전체 삭제(디렉토리/파일 unlink)
// 그 결과를 바탕으로 rebuild_block_meta()를 다시 호출
int
snapshot_delete(int id)
{
    if(id < 0){
        return -1;
    }

    char name[16], path[32];
    itoa_dec(id, name);
    safestrcpy(path, "/snapshot/", sizeof(path));
    safestrcat(path, name, sizeof(path));

    // -----
    // 1단계: /snapshot/<id> 디렉토리 트리 삭제 + 엔트리 unlink
    // -----
    begin_op();

    struct inode *snap_dir = namei(path);
    if (snap_dir == 0) {
        end_op();
        return -1;
    }

    ilock(snap_dir);
    snapshot_remove_tree(snap_dir);

    iunlock(snap_dir);
    iput(snap_dir);

    struct inode *parent;
    char base[DIRSIZ];

    parent = nameiparent(path, base); // parent = "/snapshot"
    if (parent == 0) {
        end_op();
        return -1;
    }

    ilock(parent);

```



```

struct dirent de;
uint off;
int found = 0;

for (off = 0; off < parent->size; off += sizeof(de)) {
    if (readi(parent, (char*)&de, off, sizeof(de)) != sizeof(de))
        panic("snapshot_delete: readi parent");

    if (de.inum == 0)
        continue;
    if (strcmp(de.name, base) != 0)
        continue;

    struct inode *ip = iget(parent->dev, de.inum);
    ilock(ip);
    dir_unlink_at(parent, off, ip);
    iunlock(ip);
    iput(ip);

    found = 1;
    break;
}

if (!found) {
    iunlockput(parent);
    end_op();
    return -1;
}

iunlockput(parent);

end_op(); // step1: 트리 삭제 끝

cleanup_orphan_blocks(ROOTDEV); // 비트맵과 비교해서 실제로 bfree

return 0;
}

// 디렉토리 복사용 헬퍼, 디렉토리 하나를 만드는 헬퍼
// src_dir: 원본 디렉토리 (root 또는 하위)
// dst_dir: /snapshot/<id> 안의 대응 디렉토리
// is_root: src_dir가 "/" 루트인지 여부 ("/snapshot" 제외 처리를 위해)
// src_dir: 현재 파일시스템의 디렉토리 ("/" 또는 그 하위)
// dst_dir: /snapshot/<id> 쪽의 대응 디렉토리
// is_root: src_dir가 실제 루트("/")인지 여부
// - 루트일 때만 "snapshot" 엔트리를 스킵한다.
static int
snapshot_clone_dir(struct inode *src_dir, struct inode *dst_dir, int is_root)
{
    struct dirent de;
    uint off;

    if (src_dir->type != T_DIR || dst_dir->type != T_DIR)
        panic("snapshot_clone_dir: not dir");

    for (off = 0; off < src_dir->size; off += sizeof(de)){

```

```

if(readi(src_dir, (char*)&de, off, sizeof(de)) != sizeof(de))
    panic("snapshot_clone_dir: readi");

if(de.inum == 0)
    continue;
if(strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
    continue;

// 루트("/")에서만 /snapshot 디렉토리는 캡처하지 않음
if(is_root && strcmp(de.name, "snapshot") == 0)
    continue;

// 원본 child inode
struct inode *child = iget(src_dir->dev, de.inum);
ilock(child);

if(child->type == T_DEV){
    // T_DEV는 스냅샷 대상에서 제외
    iunlockput(child);
    continue;
}

if(child->type == T_DIR){
    // 스냅샷 트리 쪽에 대응 디렉토리 하나 생성
    struct inode *newdir = create_subdir(dst_dir, de.name);
    // child, newdir 모두 ilock 상태로 재귀 호출
    snapshot_clone_dir(child, newdir, 0 /* is_root=0 */);
    iunlockput(newdir);
} else if(child->type == T_FILE){
    // 스냅샷용 파일(T_SNAP) inode 생성
    struct inode *snap_file = create_snapshot_file(dst_dir, de.name);

    // 원본 파일과 동일한 블록을 가리키도록 size/addr 복사
    snap_file->size = child->size;
    memmove(snap_file->addrs, child->addrs, sizeof(child->addrs));
    iupdate(snap_file);

    // 이 스냅샷 파일이 참조하는 모든 블록에 대해
    //   refcnt++ + snapcnt++ 반영
    inc_snap_for_inode_blocks(snap_file);

    iunlockput(snap_file);
} else {
    // 설계상 T_DIR / T_FILE / T_DEV만 나와야 한다.
    panic("snapshot_clone_dir: unexpected itype");
}

iunlockput(child);
}

return 0;
}

```

```

// parent 디렉토리 아래에 이름이 name인 새 디렉토리를 만든다.
// - parent 는 이미 ilock(parent) 가 잡혀 있어야 함.

```

```

// - begin_op()/end_op() 는 바깥에서 처리한다고 가정.
// - 성공 시: 새로 만든 디렉토리 inode* 를 "락 잡힌 상태"로 리턴.
// - 실패 상황(이미 같은 이름이 있거나 ialloc 실패 등)에서는 panic 또는 0 리턴 등으로 처리.
static struct inode *
create_subdir(struct inode *parent, char *name)
{
    struct inode *ip;

    // 혹시 같은 이름이 이미 있으면 버그이므로 panic 처리
    if((ip = dirlookup(parent, name, 0)) != 0){
        iput(ip);
        panic("create_subdir: entry already exists");
    }

    // 새 디렉토리 inode 할당
    if((ip = ialloc(parent->dev, T_DIR)) == 0)
        panic("create_subdir: ialloc");

    ilock(ip);

    ip->major = 0;
    ip->minor = 0;
    ip->nlink = 1;        // 자기 자신을 가리키는 링크('.')
    ip->size = 0;
    memset(ip->addrs, 0, sizeof(ip->addrs));
    iupdate(ip);

    // parent 의 nlink 는 "." 링크 때문에 1 증가
    parent->nlink++;
    iupdate(parent);

    // 새 디렉토리 안에 "." 과 ".." 엔트리 생성
    if(dirlink(ip, ".", ip->inum) < 0 || dirlink(ip, "..", parent->inum) < 0)
        panic("create_subdir: create dots");

    // parent 디렉토리에 이 디렉토리를 연결
    if(dirlink(parent, name, ip->inum) < 0)
        panic("create_subdir: dirlink");

    // ip 는 여전히 ilock(ip) 잡힌 상태로 리턴 (caller가 사용 후 iunlockput 등)
    return ip;
}

// parent 디렉토리 아래에 이름이 name인 일반 파일(T_FILE)을 만든다.
// - parent 는 이미 ilock(parent) 상태여야 함.
// - begin_op()/end_op() 는 바깥에서 처리한다고 가정.
// - 성공 시: 새로 만든 파일 inode* 를 "락 잡힌 상태"로 리턴.
static struct inode *
create_file(struct inode *parent, char *name)
{
    struct inode *ip;

    // 같은 이름이 이미 있으면 버그로 간주
    if((ip = dirlookup(parent, name, 0)) != 0){
        iput(ip);
        panic("create_file: entry already exists");
    }
}

```

```

// 새 inode 할당(T_FILE)
if((ip = ialloc(parent->dev, T_FILE)) == 0)
    panic("create_file: ialloc");

ilock(ip);

ip->major = 0;
ip->minor = 0;
ip->nlink = 1;    // parent에서의 한 개 링크
ip->size = 0;
memset(ip->addrs, 0, sizeof(ip->addrs));
iupdate(ip);

// parent 디렉토리에 연결
if(dirlink(parent, name, ip->inum) < 0)
    panic("create_file: dirlink");

// ip 는 여전히 ilock(ip) 상태로 리턴
return ip;
}

// parent 디렉토리 아래에 이름이 name인 스냅샷 파일(T_SNAP)을 만든다.
// - parent 는 이미 ilock(parent) 상태여야 함.
// - 데이터 블록 매핑(addrs[])는 나중에 호출자가 채울 것임.
// - 성공 시: 새 스냅샷 inode* 를 "락 잡힌 상태"로 리턴.
static struct inode *
create_snapshot_file(struct inode *parent, char *name)
{
    struct inode *ip;

    // 이미 같은 이름의 엔트리가 있으면 버그이므로 panic
    if((ip = dirlookup(parent, name, 0)) != 0){
        iput(ip);
        panic("create_snapshot_file: entry already exists");
    }

    // 새 inode 할당 (스냅샷 파일 타입)
    if((ip = ialloc(parent->dev, T_SNAP)) == 0)
        panic("create_snapshot_file: ialloc");

    ilock(ip);

    ip->major = 0;
    ip->minor = 0;
    ip->nlink = 1;    // parent 디렉토리에서의 한 개 링크
    ip->size = 0;    // 실제 size / addrs[]는 이후에 원본 파일을 보고 복사
    memset(ip->addrs, 0, sizeof(ip->addrs));
    iupdate(ip);

    // parent 디렉토리에 연결
    if(dirlink(parent, name, ip->inum) < 0)
        panic("create_snapshot_file: dirlink");

    // ip 는 ilock(ip) 상태로 리턴
    return ip;
}

```

```
// -----
// 스냅샷/롤백용 메타데이터 갱신 헬퍼
// - inc_snap_for_inode_blocks : T_SNAP이 참조하는 블록들에 대해 (refcnt++, snapcnt++)
// - dec_snap_for_inode_blocks : T_SNAP이 참조하던 블록들에 대해 (snapcnt--)
// - inc_ref_for_inode_blocks : T_FILE이 참조하는 블록들에 대해 (refcnt++)
// -----

// T_SNAP inode가 참조하는 모든 블록에 대해
// refcnt++ + snapcnt++ 수행
static void
inc_snap_for_inode_blocks(struct inode *ip)
{
    int i;
    uint b;
    struct buf *bp;
    uint *a;

    if(ip->type != T_SNAP)
        panic("inc_snap_for_inode_blocks: not T_SNAP");

    // 1) direct blocks
    for(i = 0; i < NDIRECT; i++){
        b = ip->addrs[i];
        if(b)
            blockmeta_add_ref(b, 1); // refcnt++, snapcnt++
    }

    // 2) indirect block + 그 안의 엔트리들
    if(ip->addrs[NDIRECT]){
        uint ib = ip->addrs[NDIRECT];

        // 간접 블록 자체도 하나의 블록이므로 ref/snap++
        blockmeta_add_ref(ib, 1);

        bp = bread(ip->dev, ib);
        a = (uint*)bp->data;
        for(i = 0; i < NINDIRECT; i++){
            b = a[i];
            if(b)
                blockmeta_add_ref(b, 1); // refcnt++, snapcnt++
        }
        brelse(bp);
    }
}

// T_SNAP inode가 참조하던 모든 블록에 대해
// snapcnt-- 만 수행 (refcnt는 itrunc/bdecref 에 맡김)
static void
dec_snap_for_inode_blocks(struct inode *ip)
{
    int i;
    uint b;
    struct buf *bp;
    uint *a;

    if(ip->type != T_SNAP)
        panic("dec_snap_for_inode_blocks: not T_SNAP");
}
```

```

// 1) direct blocks
for(i = 0; i < NDIRECT; i++){
    b = ip->addrs[i];
    if(b)
        blockmeta_dec_snap(b);    // snapcnt--
}

// 2) indirect block + 그 안의 엔트리들
if(ip->addrs[NDIRECT]){
    uint ib = ip->addrs[NDIRECT];

    blockmeta_dec_snap(ib);    // 간접 블록 자체

    bp = bread(ip->dev, ib);
    a = (uint*)bp->data;
    for(i = 0; i < NINDIRECT; i++){
        b = a[i];
        if(b)
            blockmeta_dec_snap(b); // snapcnt--
    }
    brelse(bp);
}
}

// T_FILE inode가 참조하는 모든 블록에 대해
//   refcnt++ 만 수행 (snapcnt는 건드리지 않음)
//
// rollback 시에 T_SNAP → T_FILE 로 복구할 때
//   새로 생긴 일반 파일이 기존 블록을 같이 쓰므로 refcnt 를 늘려주는 용도
static void
inc_ref_for_inode_blocks(struct inode *ip)
{
    int i;
    uint b;
    struct buf *bp;
    uint *a;

    if(ip->type != T_FILE)
        panic("inc_ref_for_inode_blocks: not T_FILE");

    // 1) direct blocks
    for(i = 0; i < NDIRECT; i++){
        b = ip->addrs[i];
        if(b)
            blockmeta_add_ref_nosnap(b);    // refcnt++
    }

    // 2) indirect block + 그 안의 엔트리들
    if(ip->addrs[NDIRECT]){
        uint ib = ip->addrs[NDIRECT];

        blockmeta_add_ref_nosnap(ib);    // 간접 블록 자체도 refcnt++

        bp = bread(ip->dev, ib);
        a = (uint*)bp->data;
        for(i = 0; i < NINDIRECT; i++){
            b = a[i];
            if(b)

```

```

        blockmeta_add_ref_nosnap(b); // refcnt++
    }
    brelse(bp);
}
}

// 트리 삭제 헬퍼(snapshot_delect(id)의 헬퍼)
// dir: 이미 ilock(dir) 상태여야 함
// 역할: dir 아래의 모든 엔트리를 재귀적으로 삭제 (스냅샷 트리 전체 제거)
//      단, '.' / '..' 은 건드리지 않음.
//      dir 자신은 여기서 unlink하지 않고, snapshot_delete() 쪽에서
//      부모 디렉토리에서 지워준다.
static int
snapshot_remove_tree(struct inode *dir)
{
    struct dirent de;
    uint off;

    if(dir->type != T_DIR)
        panic("snapshot_remove_tree: not dir");

    // dir의 dirent들을 처음부터 끝까지 스캔
    for(off = 0; off < dir->size; off += sizeof(de)){
        if(readi(dir, (char*)&de, off, sizeof(de)) != sizeof(de))
            panic("snapshot_remove_tree: readi");

        // 빈 엔트리면 넘어감
        if(de.inum == 0)
            continue;

        // "." / ".." 는 건드리지 않음
        if(strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
            continue;

        // 자식 inode 얻기
        struct inode *child = iget(dir->dev, de.inum);
        ilock(child);

        if(child->type == T_DIR){
            // 하위 디렉토리라면, 그 안을 먼저 재귀적으로 싹 비운다.
            snapshot_remove_tree(child);
            // 그리고 현재 dir에서 이 디렉토리 엔트리를 제거
            dir_unlink_at(dir, off, child);
            iunlock(child);
            iput(child);
        } else {
            // 스냅샷 파일(T_SNAP)이면 먼저 snapcnt-- 반영
            if(child->type == T_SNAP){
                dec_snap_for_inode_blocks(child);
            }

            // 그 다음 디렉토리 엔트리 unlink + nlink 감소
            dir_unlink_at(dir, off, child);
            iunlock(child);
            iput(child);
        }
    }
}

```

```

// dir 자체는 여기서 제거하지 않는다.
// /snapshot/<id> 디렉토리 자체를 없애는 건 snapshot_delete() 가
// 부모 디렉토리(/snapshot)에서 이름 <id> 를 unlink하면서 처리.
return 0;
}

// dp: 부모 디렉토리 (락 잡힌 상태여야 함)
// off: dp 내에서 이 엔트리가 위치한 바이트 오프셋
// ip: 이 엔트리가 가리키는 자식 inode (락 잡힌 상태여야 함)
//
// 역할:
// - dp의 off 위치에 있는 dirent를 비우고 (de.inum = 0)
// - 디렉토리라면 dp->nlink--
// - ip->nlink-- 하고 iupdate(ip)
// - 락 해제/ iput 은 caller가 한다 (여기서는 하지 않음)
static void
dir_unlink_at(struct inode *dp, uint off, struct inode *ip)
{
    struct dirent de;

    if(dp->type != T_DIR)
        panic("dir_unlink_at: dp not dir");

    // dirent 비우기
    memset(&de, 0, sizeof(de));
    if(writei(dp, (char*)&de, off, sizeof(de)) != sizeof(de))
        panic("dir_unlink_at: writei");

    // 디렉토리였으면 부모 링크 감소 ( "." 제거 효과 )
    if(ip->type == T_DIR){
        if(dp->nlink < 1)
            panic("dir_unlink_at: dp nlink < 1");
        dp->nlink--;
        iupdate(dp);
    }

    // 자식 inode의 링크 감소
    if(ip->nlink < 1)
        panic("dir_unlink_at: ip nlink < 1");
    ip->nlink--;
    iupdate(ip);
}

// 루프 디렉토리 아래의 모든 파일/디렉토리(snapshot 제외)를 정리하는 헬퍼 함수
// 루트 디렉토리("/")에서
// - ".", ".." 는 건드리지 않고
// - "snapshot" 디렉토리는 남겨두고
// - 나머지 모든 엔트리를 삭제(파일/디렉토리 전부 재귀 삭제)
//
// 호출 전후에 begin_op()/end_op() 는 바깥에서 처리한다고 가정.
static int
clear_root_except_snapshot(void)
{
    struct inode *root = iget(ROOTDEV, ROOTINO);

```



```

ilock(root);

struct dirent de;
uint off;

for(off = 0; off < root->size; off += sizeof(de)){
    if(readi(root, (char*)&de, off, sizeof(de)) != sizeof(de))
        panic("clear_root: readi");

    if(de.inum == 0)
        continue;

    // ".", ".."는 삭제 금지
    if(strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
        continue;

    // "/snapshot" 디렉토리는 롤백 시에도 남겨둔다
    if(strcmp(de.name, "snapshot") == 0)
        continue;

    // 이제 이 엔트리는 제거 대상
    struct inode *child = iget(root->dev, de.inum);
    ilock(child);

    if(child->type == T_DIR){
        // 하위 디렉토리면, 그 안을 먼저 싹 비운다 (스냅샷이 아니어도 일반 디렉토리)
        snapshot_remove_tree(child);
        // 그리고 root 에서 이 디렉토리 엔트리를 제거
        dir_unlink_at(root, off, child);
        iunlock(child);
        iput(child);
    } else {
        // 일반 파일/장치 파일(T_FILE, T_DEV 등)인 경우, 바로 unlink
        dir_unlink_at(root, off, child);
        iunlock(child);
        iput(child);
    }
}

iunlockput(root);
return 0;
}

// src_snap_dir: /snapshot/<id> 트리 안의 디렉토리
// dst_dir      : 현재 파일시스템 쪽 디렉토리 ("/" 또는 그 하위)
// 역할:
//   - src_snap_dir의 내용을 그대로 dst_dir 아래에 재귀적으로 생성한다.
//   - 스냅샷 디렉토리(T_DIR)는 일반 T_DIR로,
//   - 스냅샷 파일(T_SNAP)은 일반 T_FILE로 복구하되,
//   data block(addrsl[])는 그대로 공유해서 COW 조건만 만족시키도록 한다.
static int
rollback_clone_dir(struct inode *src_snap_dir, struct inode *dst_dir)
{
    struct dirent de;
    uint off;

    if(src_snap_dir->type != T_DIR || dst_dir->type != T_DIR)
        panic("rollback_clone_dir: not dir");

```

```

for(off = 0; off < src_snap_dir->size; off += sizeof(de)){
    if(readi(src_snap_dir, (char*)&de, off, sizeof(de)) != sizeof(de))
        panic("rollback_clone_dir: readi");

    if(de.inum == 0)
        continue;
    if(strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
        continue;

    struct inode *src = iget(src_snap_dir->dev, de.inum);
    ilock(src);

    if(src->type == T_DIR){
        // 새 디렉토리(T_DIR) 생성
        struct inode *newdir = create_subdir(dst_dir, de.name);
        // 양쪽 디렉토리는 ilock 상태에서 재귀
        rollback_clone_dir(src, newdir);
        iunlockput(newdir);
    } else if(src->type == T_SNAP){
        // 현재 파일시스템용 일반 파일(T_FILE) 생성
        struct inode *file = create_file(dst_dir, de.name);

        file->size = src->size;
        memmove(file->addrs, src->addrs, sizeof(file->addrs));
        iupdate(file);

        // 새로 생긴 일반 파일이 같은 블록을 같이 쓰므로
        // 해당 블록들의 refcnt++ (snapcnt는 그대로)
        inc_ref_for_inode_blocks(file);

        iunlockput(file);
    } else {
        // 설계상 스냅샷 트리에는 T_DIR / T_SNAP만 있어야 함
        panic("rollback_clone_dir: unexpected itype");
    }

    iunlockput(src);
}

return 0;
}

// 유저 프로그램 print_addr를 위한 시스템콜
int
sys_print_addr(void)
{
    char *path;
    if(argstr(0, &path) < 0)
        return -1;

    begin_op();
    struct inode *ip = namei(path);
    if(ip == 0){
        end_op();
        cprintf("print_addr: no such file %s\n", path);
        return -1;
    }
}

```

```

ilock(ip);

if(ip->type != T_FILE && ip->type != T_SNAP){
    cprintf("print_addr: not a regular file (type=%d)\n", ip->type);
    iunlockput(ip);
    end_op();
    return -1;
}

// 1) direct block 주소들 출력
int i;
for(i = 0; i < NDIRECT; i++){
    if(ip->addrs[i] == 0)
        break;
    cprintf("addr[%d] : %x\n", i, ip->addrs[i]);
}

// 2) indirect pointer + 첫 번째 indirect 엔트리 예시 출력
if(ip->addrs[NDIRECT]){
    uint indb = ip->addrs[NDIRECT];
    cprintf("addr[%d] : %x (INDIRECT POINTER)\n", NDIRECT, indb);

    struct buf *bp = bread(ip->dev, indb);
    uint *a = (uint*)bp->data;

    if(a[0] != 0){
        cprintf("addr[%d] -> [0] (bn : %d) : %x\n",
            NDIRECT, NDIRECT, a[0]);
    }
    brelse(bp);
}

iunlockput(ip);
end_op();
return 0;
}

```

18. sysproc.c

```

#include "types.h"
#include "x86.h"
#include "defs.h"
#include "date.h"
#include "param.h"
#include "memlayout.h"
#include "mmu.h"
#include "proc.h"
#include "fs.h"
#include "fcntl.h"

extern int snapshot_create(void);
extern int snapshot_rollback(int id);
extern int snapshot_delete(int id);

int
sys_fork(void)
{
    return fork();
}

```

```

int
sys_exit(void)
{
    exit();
    return 0; // not reached
}

```

```

int
sys_wait(void)
{
    return wait();
}

```

```

int
sys_kill(void)
{
    int pid;

    if(argint(0, &pid) < 0)
        return -1;
    return kill(pid);
}

```

```

int
sys_getpid(void)
{
    return myproc()->pid;
}

```

```

int
sys_sbrk(void)
{
    int addr;
    int n;

    if(argint(0, &n) < 0)
        return -1;
    addr = myproc()->sz;
    if(growproc(n) < 0)
        return -1;
    return addr;
}

```

```

int
sys_sleep(void)
{
    int n;
    uint ticks0;

    if(argint(0, &n) < 0)
        return -1;
    acquire(&tickslock);
    ticks0 = ticks;
    while(ticks - ticks0 < n){
        if(myproc()->killed){
            release(&tickslock);
            return -1;

```

```

    }
    sleep(&ticks, &tickslock);
}
release(&tickslock);
return 0;
}

// return how many clock tick interrupts have occurred
// since start.
int
sys_uptime(void)
{
    uint xticks;

    acquire(&tickslock);
    xticks = ticks;
    release(&tickslock);
    return xticks;
}

// snapshot_create()를 호출하기 위한 시스템콜 래퍼
int
sys_snapshot_create(void)
{
    return snapshot_create();
}

// snapshot_rollback()을 호출하기 위한 시스템콜 래퍼
int
sys_snapshot_rollback(void)
{
    int id;
    if (argint(0, &id) < 0)
        return -1;
    return snapshot_rollback(id);
}

// snapshot_delete()를 호출하기 위한 시스템콜 래퍼
int
sys_snapshot_delete(void)
{
    int id;
    if (argint(0, &id) < 0)
        return -1;
    return snapshot_delete(id);
}
}

19. user.h
struct stat;
struct rtcdate;

// system calls
int fork(void);
int exit(void) __attribute__((noreturn));
int wait(void);
int pipe(int*);
int write(int, const void*, int);
int read(int, void*, int);
int close(int);

```

```

int kill(int);
int exec(char*, char**);
int open(const char*, int);
int mknod(const char*, short, short);
int unlink(const char*);
int fstat(int fd, struct stat*);
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int getpid(void);
char* sbrk(int);
int sleep(int);
int uptime(void);

```

// 설계4 구현한 스냅샷 관련 함수

```

int snapshot_create(void);
int snapshot_rollback(int);
int snapshot_delete(int);

```

// 블록 주소 출력

```

int print_addr(char *);

```

// ulib.c

```

int stat(const char*, struct stat*);
char* strcpy(char*, const char*);
void *memmove(void*, const void*, int);
char* strchr(const char*, char c);
int strcmp(const char*, const char*);
void printf(int, const char*, ...);
char* gets(char*, int max);
uint strlen(const char*);
void* memset(void*, int, uint);
void* malloc(uint);
void free(void*);
int atoi(const char*);

```

20. usys.S

```

#include "syscall.h"

```

```

#include "traps.h"

```

```

#define SYSCALL(name) \
    .globl name; \
    name: \
        movl $SYS_ ## name, %eax; \
        int $T_SYSCALL; \
        ret

```

```

SYSCALL(fork)

```

```

SYSCALL(exit)

```

```

SYSCALL(wait)

```

```

SYSCALL(pipe)

```

```

SYSCALL(read)

```

```

SYSCALL(write)

```

```

SYSCALL(close)

```

```

SYSCALL(kill)

```

```

SYSCALL(exec)

```

```

SYSCALL(open)

```

SYSCALL(mknod)
SYSCALL(unlink)
SYSCALL(fstat)
SYSCALL(link)
SYSCALL(mkdir)
SYSCALL(chdir)
SYSCALL(dup)
SYSCALL(getpid)
SYSCALL(sbrk)
SYSCALL(sleep)
SYSCALL(uptime)
SYSCALL(snapshot_create)
SYSCALL(snapshot_rollback)
SYSCALL(snapshot_delete)
SYSCALL(print_addr)